



Unity를 이용한 2D 게임프로그래밍

Lecture 2

Unity 게임 만들기

강영민

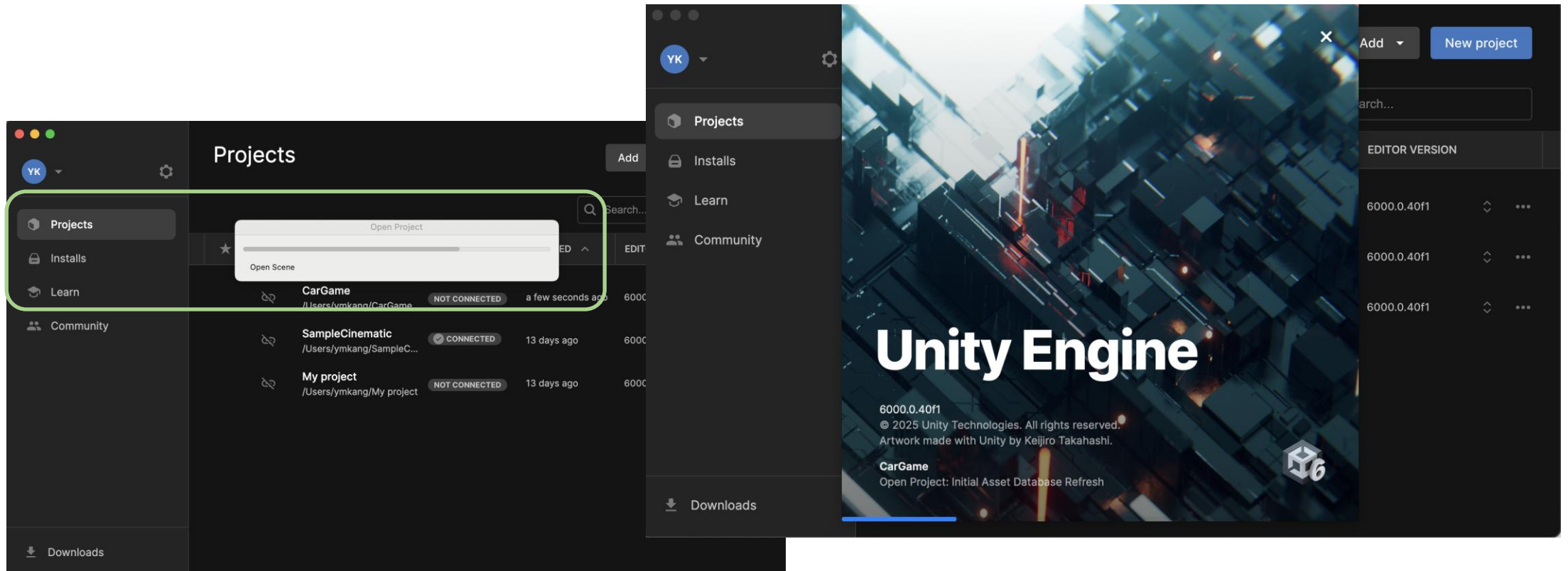
동명대학교 게임학부

학습목표

- 유니티로 간단한 게임을 만들어 본다.
- 입력 시스템은 왜 필요한지 이해한다.
- 입력 시스템을 이용하여 캐릭터를 제어해 본다.
- 콜라이더와 트리거를 이해한다.
- 물리적 동작이 일어나게 한다.
- 게임 배경을 만들어 본다.
- 게임 캐릭터를 따라 카메라가 이동하게 한다.

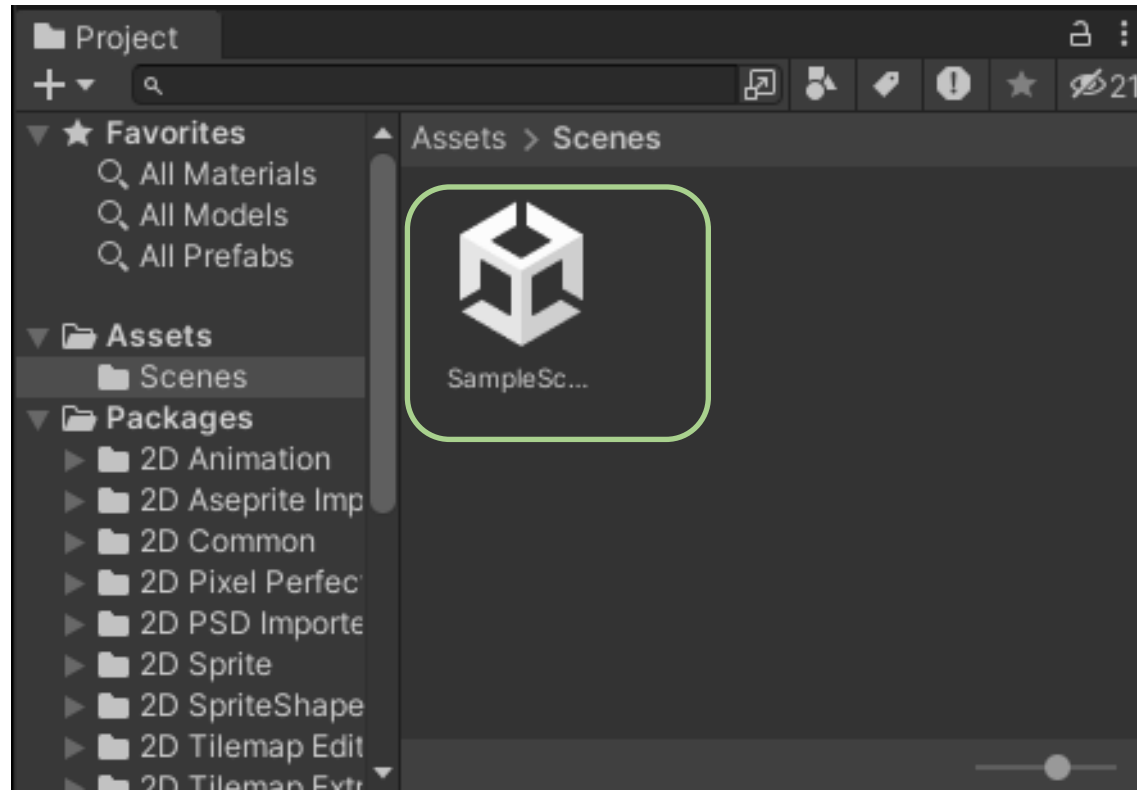
Unity Hub로 새로운 프로젝트를 생성하자

- CarGame으로 생성



기본 장면 – SampleScene

- 유니티 프로젝트를 생성하면 SampleScene이 생성됨

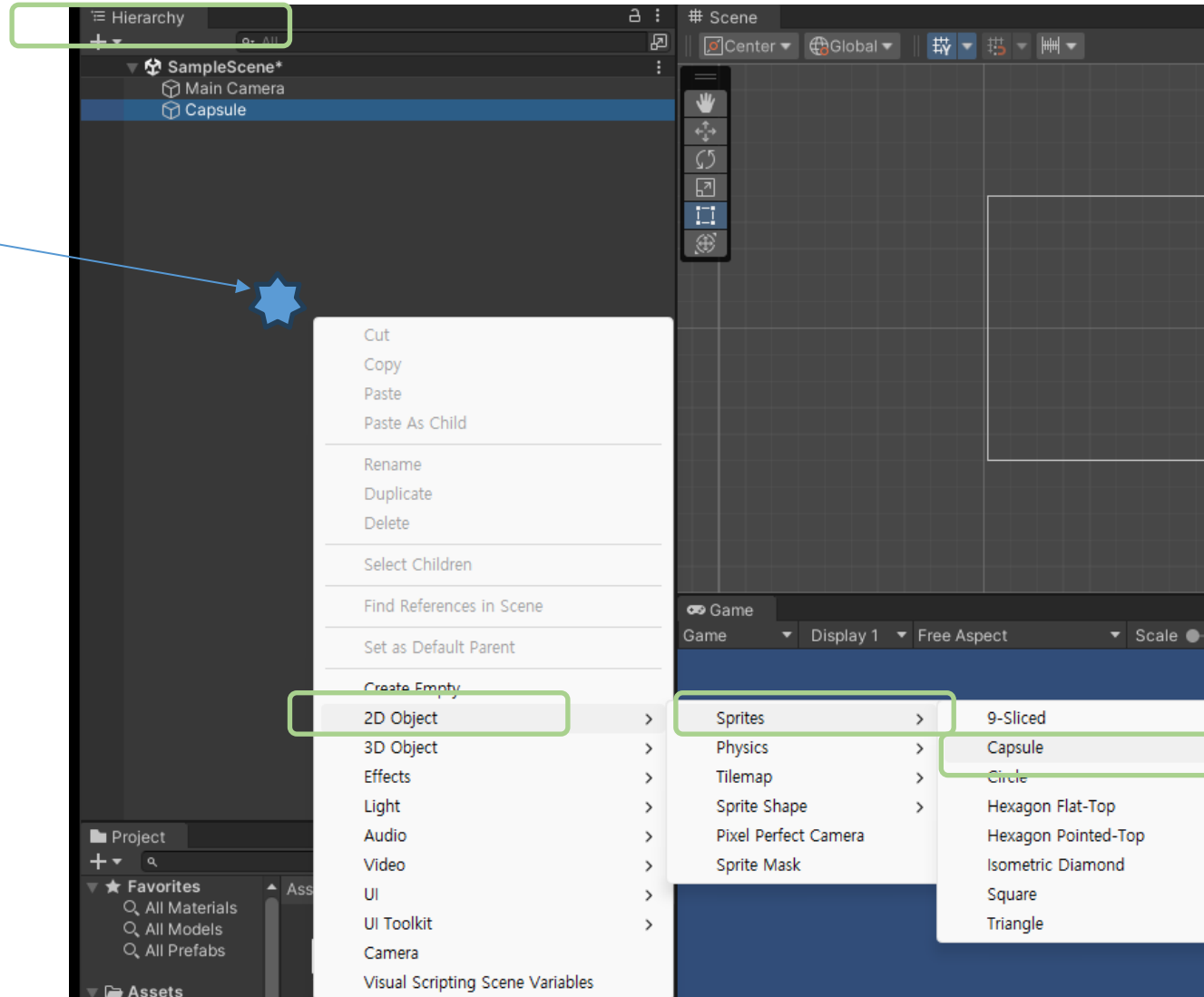


(다시) 간단한 객체를 넣고 움직여 보자

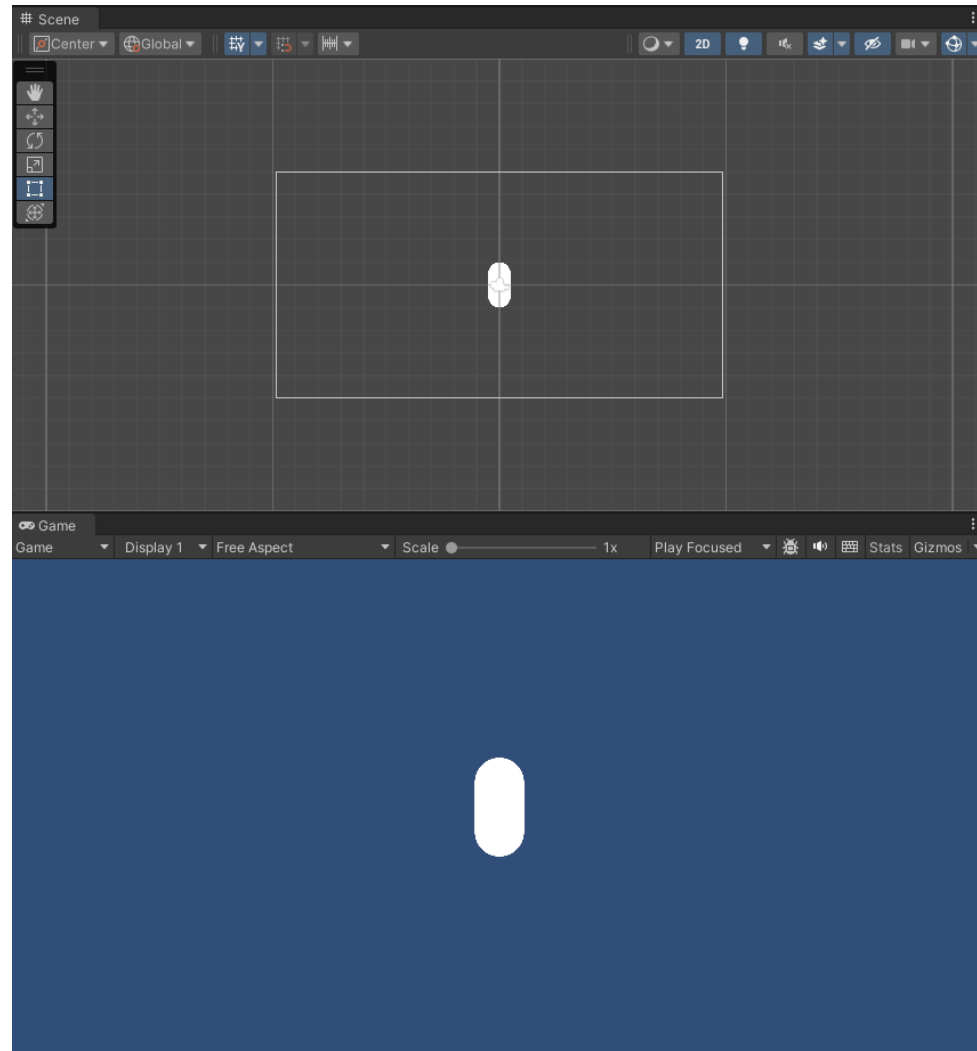
- 2D 게임 → 스프라이트 객체 사용
 - 캡슐 스프라이트 추가
 - 캡슐 스프라이트에 C# 코드 연결
 - 코딩을 통해 스프라이트 움직이기

캡슐 스프라이트 추가

Right click!



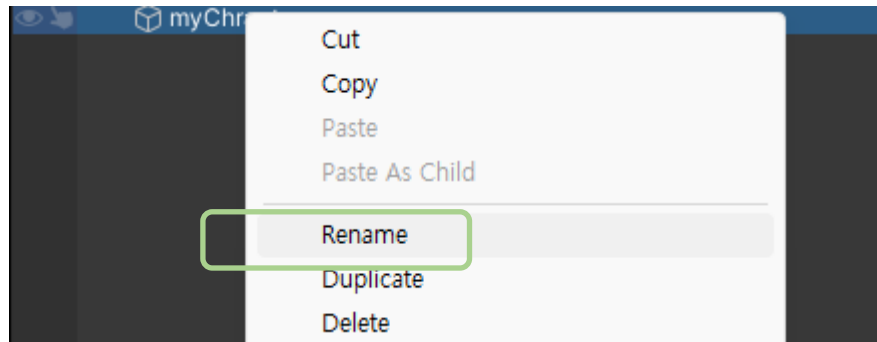
캡슐 스프라이트 확인



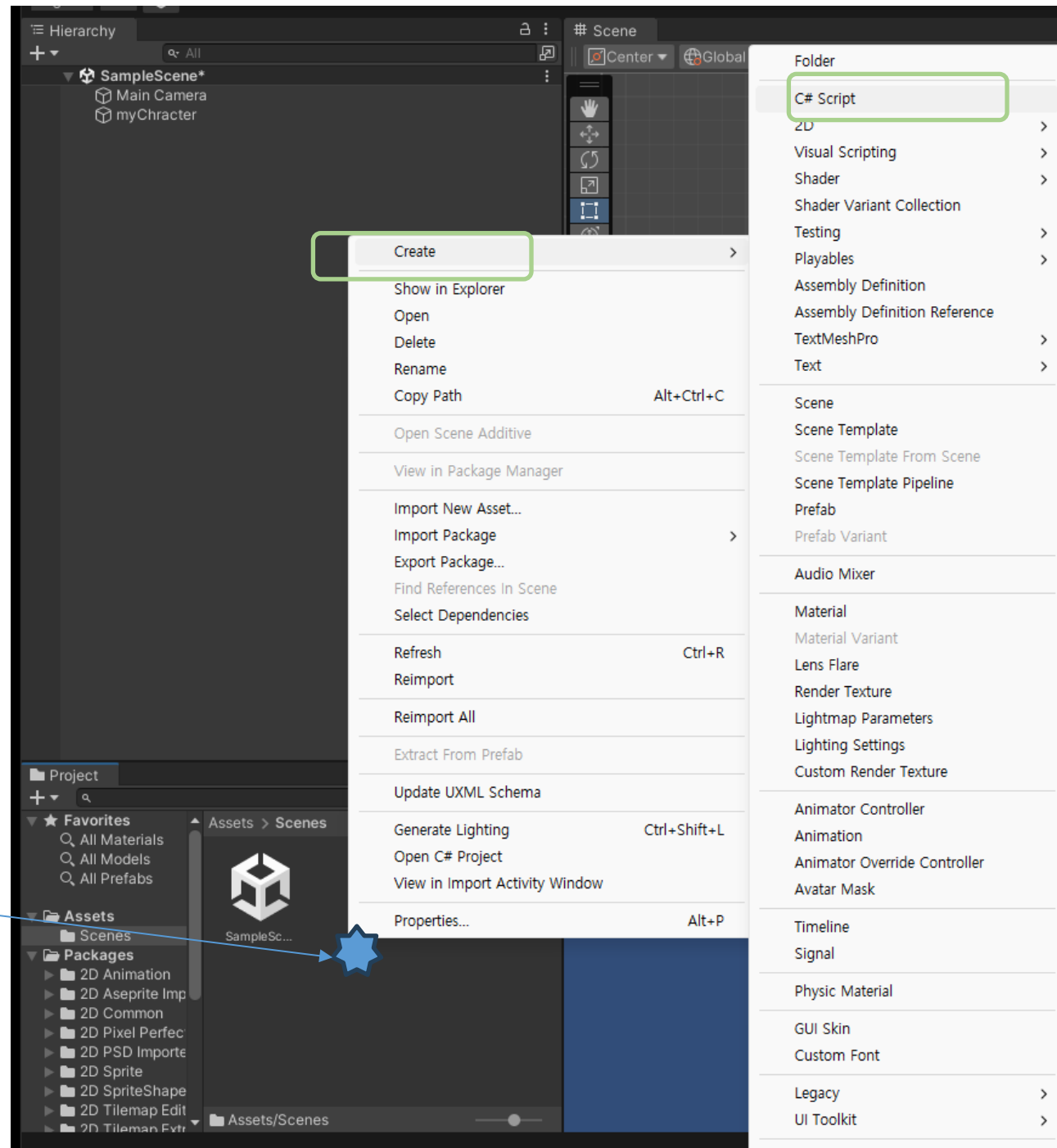
캡슐 스프라이트 움직이기 위한 “method” 사용

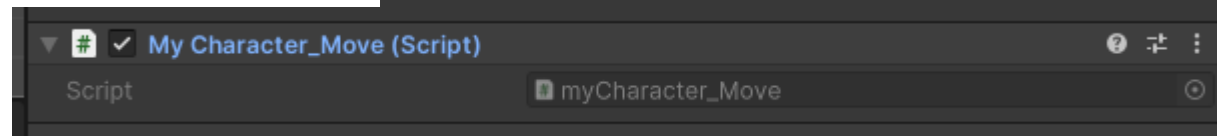
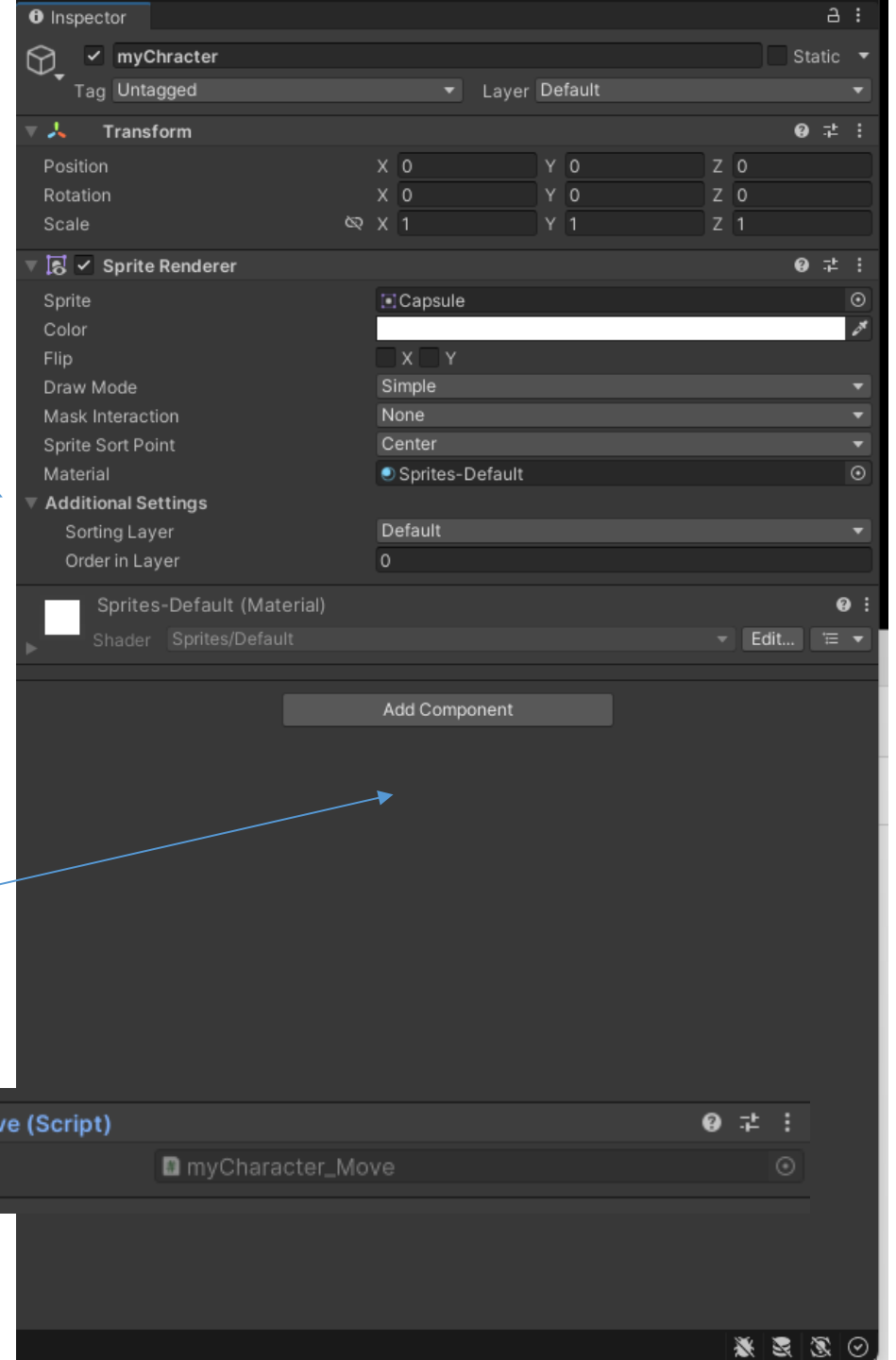
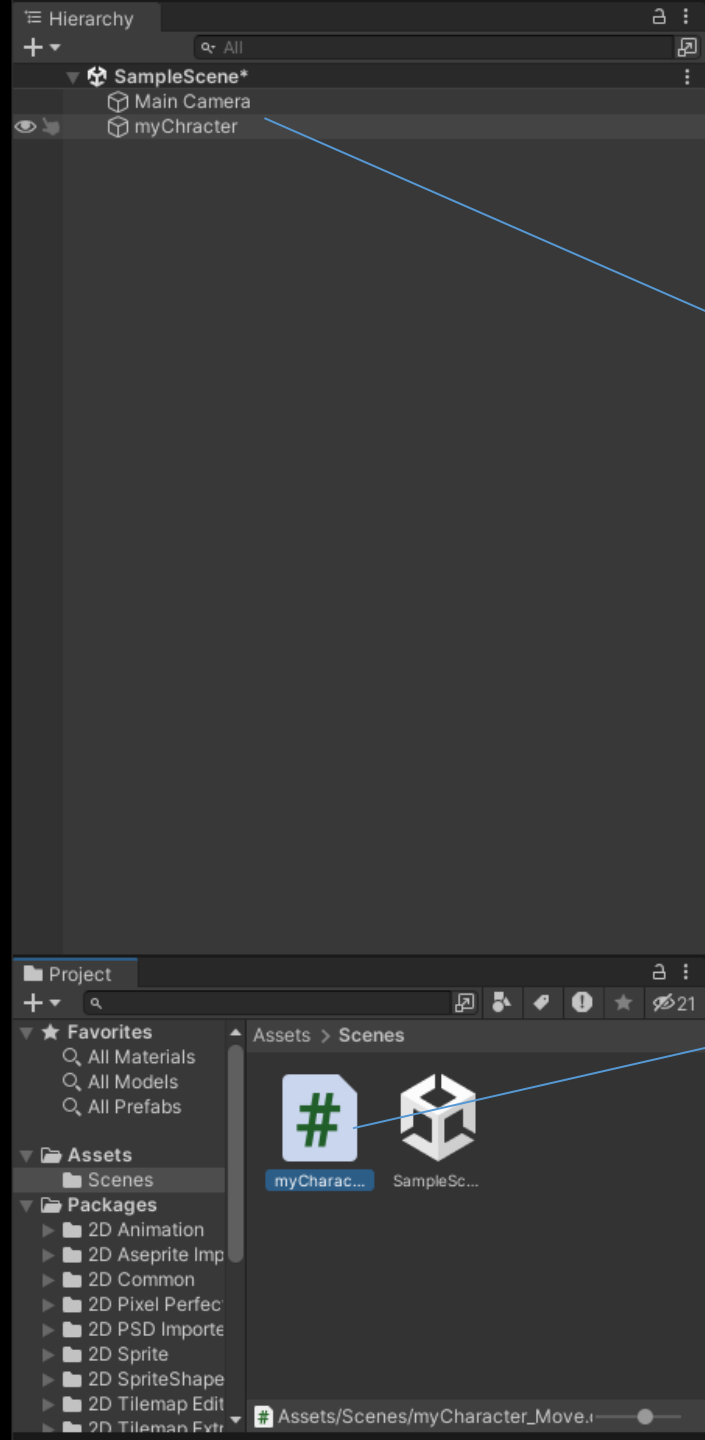
- 유니티의 메소드
 - 우리의 게임이 무엇인가를 할 수 있도록 하는 코드의 덩어리를 실행시키는 도구
 - 함수와 비슷한 개념 - 메소드는 객체에 딸린 함수
- 사용가능한 메소드
 - 유니티에서 제공하는 메소드
 - 우리가 직접 만든 메소드
- 메소드를 만들고 실행하는 것
 - 만들기: 무엇인가를 할 수 있는 코드를 메소드로 작성해 두는 것
 - 실행: 만들어 놓은 메소드를 호출하여 해당 동작이 이루어지게 하는 것

스크립트를 연결하기



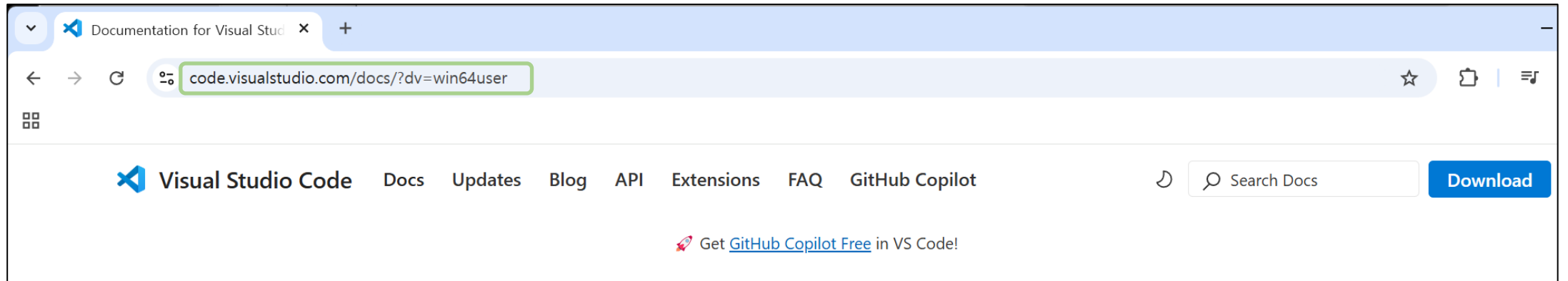
Right click!





스크립트를 수정해 보자

- 스크립트를 두 번 클릭
 - 연결된 외부 개발 도구가 열림
 - Visual Studio Code를 연결해 두었음
 - 열리지 않는 경우에는 도구가 설치되지 않은 상태임
- 필요한 경우 설치



필요한 Extension

The screenshot shows the Visual Studio Code interface with the C# Dev Kit extension installed. The left sidebar displays the 'EXTENSIONS: MARKETPLACE' view, where the C# Dev Kit extension is highlighted with a green box. The main editor area shows the 'C# Dev Kit' extension details page, which includes the extension's icon, name, publisher (Microsoft), version (8,444,992), and a 4.5-star rating (157 reviews). The page also features buttons for 'Disable', 'Uninstall', and 'Switch to Pre-Release Version', along with an 'Auto Update' checkbox. The 'C# Dev Kit for Visual Studio Code' section describes the extension's purpose and lists its features, including project and solution management, native testing environment, and Roslyn-powered language service. A 'Quick Start' section provides three steps for installation and setup, and a 'Feature List & Walkthrough' section lists the extension's capabilities.

EXTENSIONS: MARKETPLACE

- Mintlify Doc Writer for Python, JavaScript, T... 12ms
AI powered documentation writer for JavaScript, Python...
- C# Dev Kit**
Official C# extension from Microsoft
Microsoft
- C#
Base language support for C#
Microsoft
- IntelliCode for C# Dev Kit
AI-assisted development for C# Dev Kit
Microsoft
- [DEPRECATED] C# Format Revolve
[DEPRECATED] Fix format of usings / indents / braces
Przemysław Orłowski
- C# Extensions
C# IDE Extensions for VSCode
jchannon
- C# Extensions
C# IDE Extensions for VSCode
JosKreativ
- C# XML Documentation Comments
Generate C# XML documentation comments for ///
Keisuke Kato
- C# Snippets
C# Snippets for Visual Studio Code
Jorge Serrano
- Auto-Using for C#
Provides intellisense for and imports references from al...
Fudge
- C# Namespace Autocompletion

C# Dev Kit
Microsoft | microsoft.com | 8,444,992 | ★★★★★ (157)
Official C# extension from Microsoft

Disable | Uninstall | Switch to Pre-Release Version | Auto Update

C# Dev Kit for Visual Studio Code

C# Dev Kit helps you manage your code with a solution explorer and test your code with integrated unit test discovery and execution, elevating your C# development experience wherever you like to develop (Windows, macOS, Linux, and even in a Codespace).

This extension builds on top of the great C# language capabilities provided by the C# extension and enhances your C# environment by adding a set of powerful tools and utilities that integrate natively with VS Code to help C# developers write, debug, and maintain their code faster and with fewer errors. Some of this new tooling includes but is not limited to:

- C# project and solution management via an integrated solution explorer
- Native testing environment to run and debug tests using the Test Explorer
- Roslyn-powered language service for best in-class C# language features such as code navigation, refactoring, semantic awareness, and more

Quick Start

- Install C# Dev Kit (The C# extension and the .NET Install Tool will automatically be installed).
- Open a folder/workspace that contains a C# project (.csproj) and the extension will activate, or use the ".NET: New Project..." command to create a new C# project.
- Check out the C# Getting Started documentation to learn more!

Feature List & Walkthrough

- Project System / Solution Explorer
 - Solution Node Actions
 - Add Project
 - Build/Run Project

필요한 Extension

The screenshot displays the Visual Studio Code interface with the Unity extension marketplace open. The left sidebar shows a list of extensions, with 'Unity' highlighted. The main panel shows the details for the 'Unity' extension by Microsoft, which integrates Visual Studio Code with Unity. The extension is currently installed, and the 'Uninstall' button is visible. Below the extension details, there is a section titled 'Unity for Visual Studio Code' which describes the extension's purpose and lists its features.

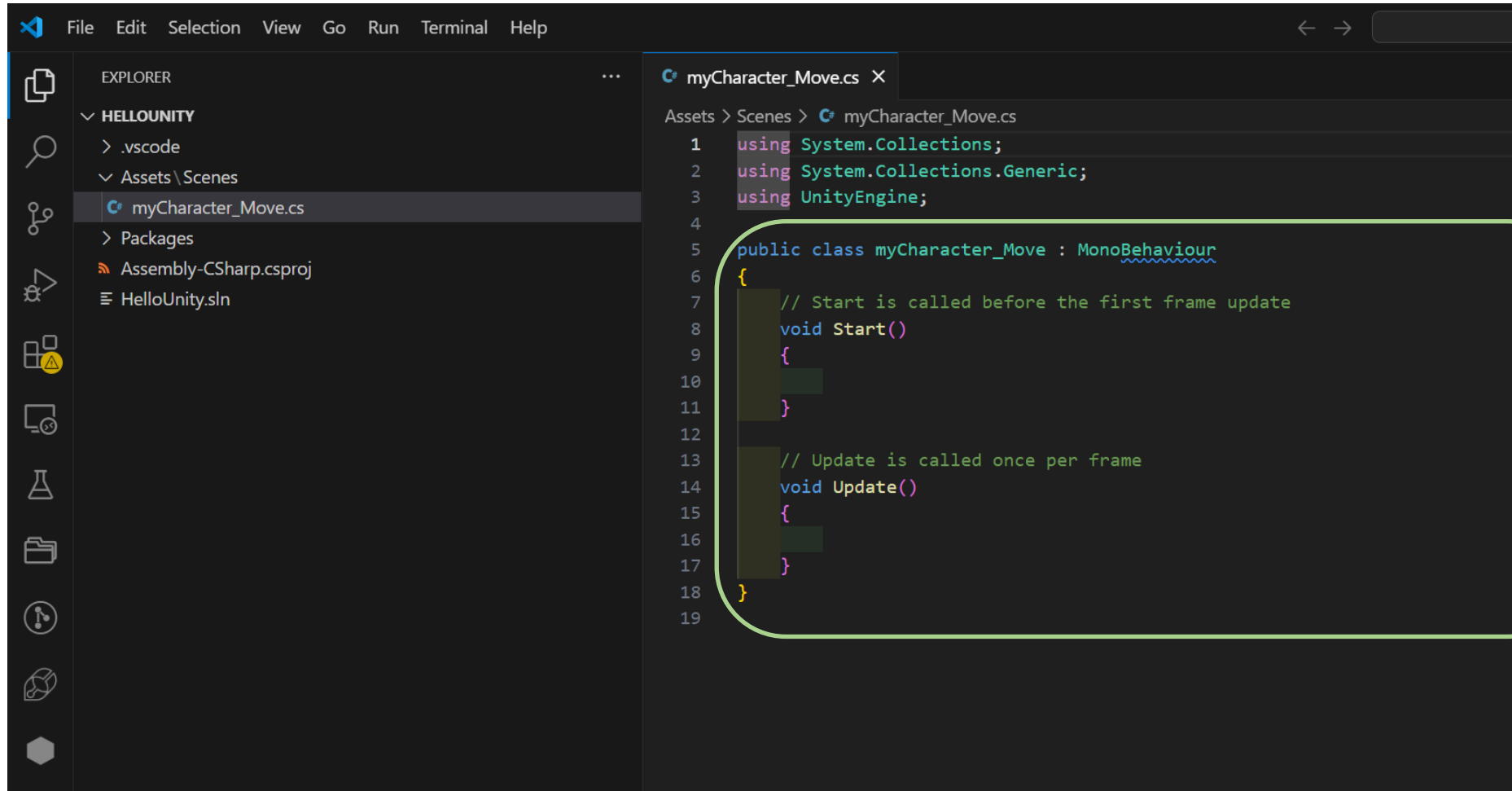
Unity for Visual Studio Code

The Unity extension provides C# developers with a lightweight and streamlined [Unity](#) development experience on Visual Studio Code. It builds on top of the rich C# capabilities provided by the [C# Dev Kit](#) and [C#](#) extensions. This extension integrates natively with Visual Studio Code and includes multiple productivity features including:

- A Unity debugger to debug your Unity editor and your Unity players.
- Unity specific C# analyzers and refactorings.
- Code coloration for Unity file formats (.asmdef, .shader, .uss, .uxml).

The bottom of the image shows a preview of the Unity editor running a game, with the Unity Inspector and Hierarchy panels visible.

코드 편집



```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class myCharacter_Move : MonoBehaviour
6 {
7     // Start is called before the first frame update
8     void Start()
9     {
10
11     }
12
13     // Update is called once per frame
14     void Update()
15     {
16
17     }
18 }
19
```

이 클래스가
우리의 캐릭터에
연결되어 있음

두 개의 클래스
Start, Update는
자동으로 호출되는
메소드

Start: 플레이될 때
Update: 매 프레임

코드 편집 – SerializeFiled, transform

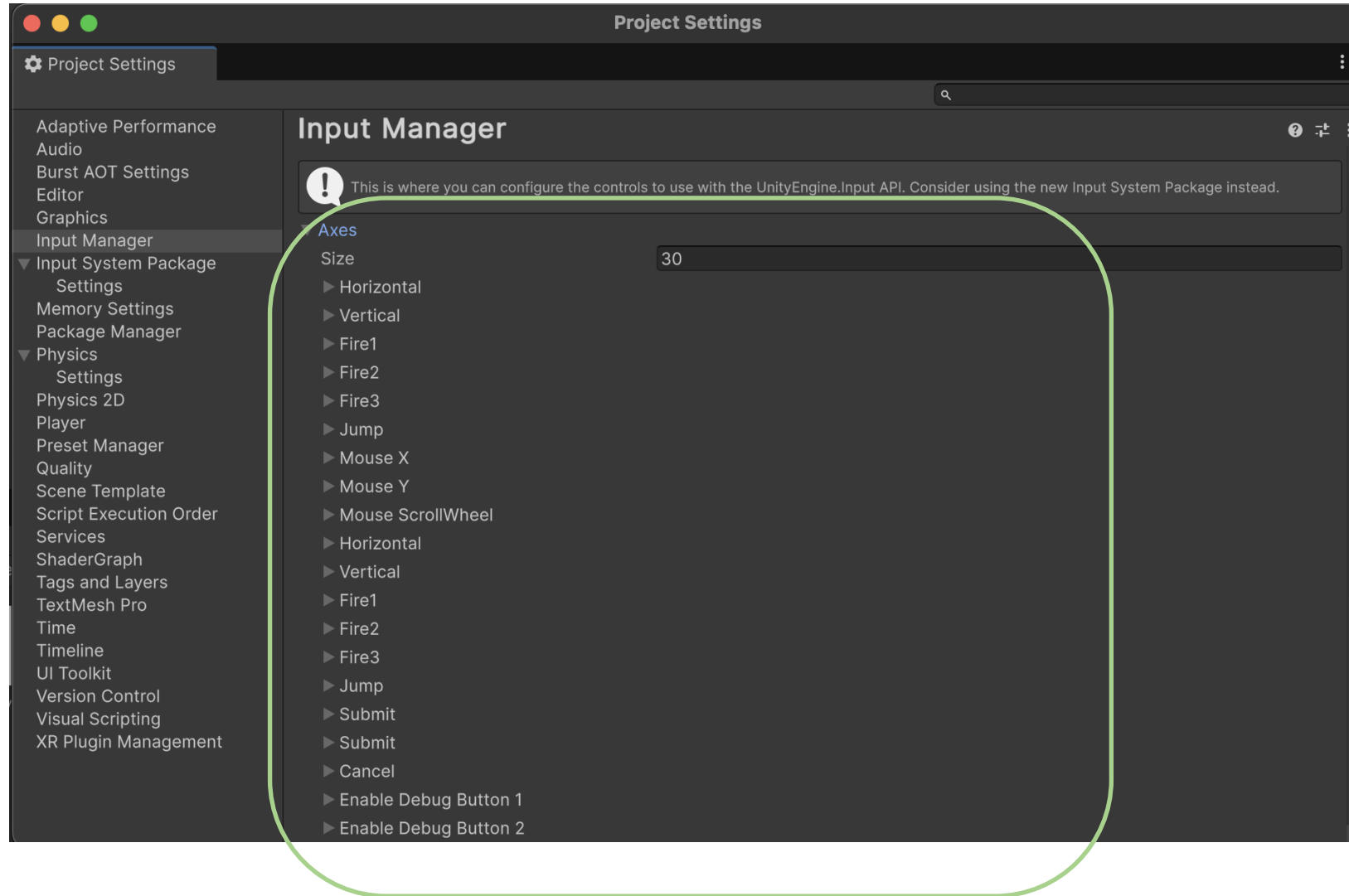
```
myCharacter_move.cs ×
Users > ymkang > CarGame > Assets > Scripts > myCharacter_move.cs > myCharacter_move > Update
1  using UnityEngine;
2
3  0 references
4  public class myCharacter_move : MonoBehaviour
5  {
6      1 reference
7      [SerializeField] float steerSpeed = 1f;
8      1 reference
9      [SerializeField] float moveSpeed = 0.01f;
10     // Start is called once before the first execution of Update after the MonoBehaviour is created
11     0 references
12     void Start()
13     {
14
15
16
17     // Update is called once per frame
18     0 references
19     void Update()
20     {
21         transform.Rotate(0, 0, steerSpeed);
22         transform.Translate(0, moveSpeed, 0);
23     }
24 }
```

제자리에서 돌기

Input System 사용하기

- 입력 시스템: 사용자의 물리적 동작을 게임에 필요한 정보로 변경
 - 물리적 동작: 버튼 누르기, 키 누르기, 조이스틱 조작 등
- 현재의 유니티는 두 종류의 입력 시스템을 가짐
 - Old system
 - New system
- 이 프로젝트에서는 old system을 사용해 보자

Input System 확인



Edit → Project
Settings

Input Manager
→
다양한 Axis

Input System 사용시 Axis의 정확한 이름 필요

The screenshot shows the Unity Input Manager interface. It is divided into two sections: Horizontal and Vertical. Each section has a list of properties on the left and their corresponding values on the right. The 'Name' property for both axes is highlighted with a green box.

Property	Horizontal Value	Vertical Value
Name	Horizontal	Vertical
Descriptive Name		
Descriptive Negative Name		
Negative Button	left	down
Positive Button	right	up
Alt Negative Button	a	s
Alt Positive Button	d	w
Gravity	3	3
Dead	0.001	
Sensitivity	3	
Snap	☒	
Invert	☐	
Type	Key or Mouse Button	
Axis	X axis	
Joy Num	Get Motion from all Joysticks	

Axis의 신호를 스크립트에서 얻을 수 있음

`Input.GetAxis(name)`

코드 편집 – SerializeField, transform

```
myCharacter_move.cs ×
Users > ymkang > CarGame > Assets > Scripts > myCharacter_move.cs > myCharacter_move > Update
1  using UnityEngine;
2
3  0 references
4  public class myCharacter_move : MonoBehaviour
5  {
6      1 reference
7      [SerializeField] float steerSpeed = 1f;
8      1 reference
9      [SerializeField] float moveSpeed = 0.01f;
10     // Start is called once before the first execution of Update after the MonoBehaviour is created
11     0 references
12     void Start()
13     {
14
15     }
16
17     // Update is called once per frame
18     0 references
19     void Update()
20     {
21
22         transform.Rotate(0, 0, steerSpeed);
23         transform.Translate(0, moveSpeed, 0);
24     }
25 }
```

입력을 통해 캐릭터 제어

```
// Update is called once per frame
0 references
void Update()
{

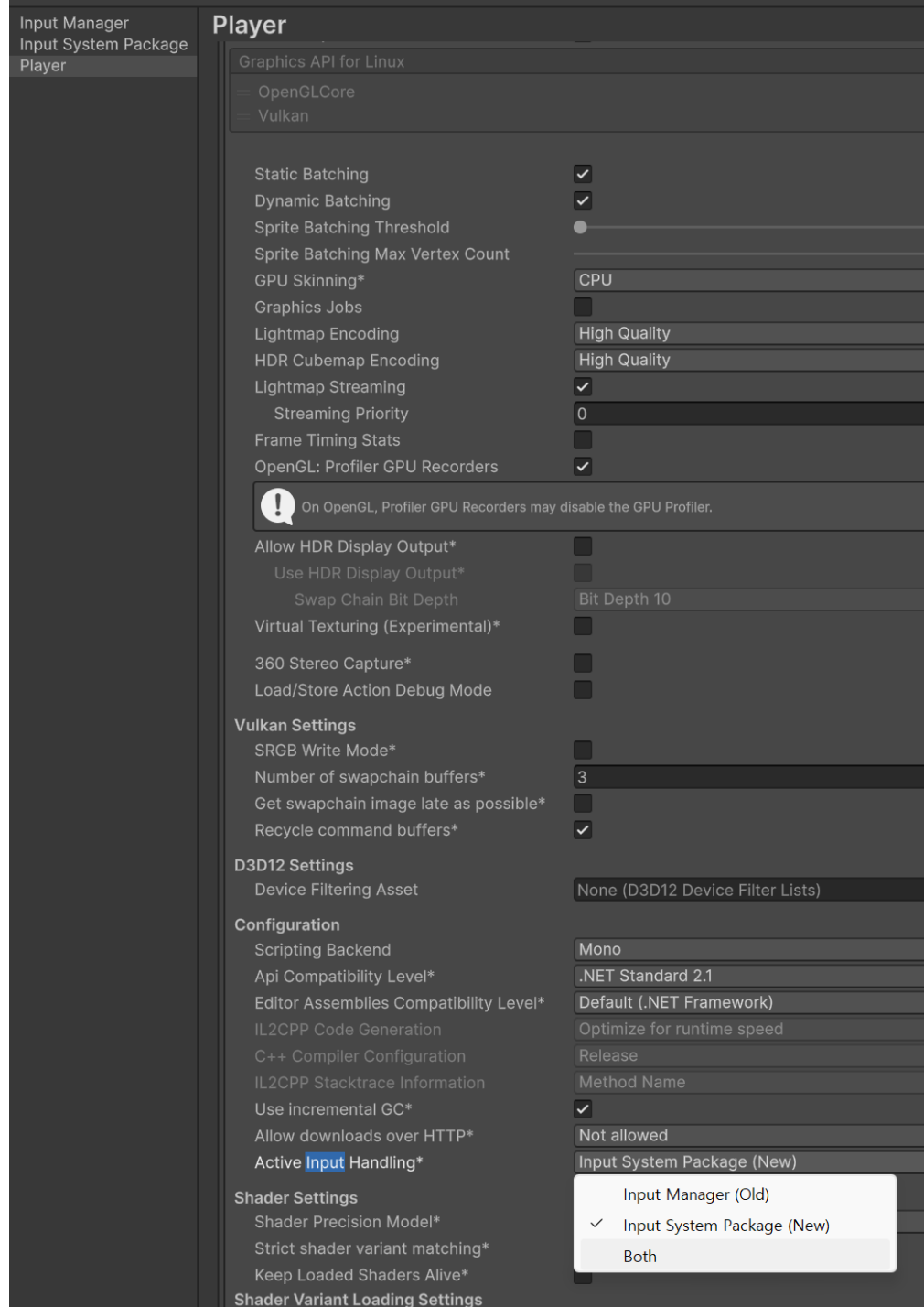
    float steerAmount = -Input.GetAxis("Horizontal");
    transform.Rotate(0, 0, steerSpeed * steerAmount);
    transform.Translate(0, moveSpeed, 0);
}
```

캐릭터 제어

게임이란 무엇이고?
입력시스템의 역할은?

```
C# myCharacter_move.cs X
Users > ymkang > CarGame > Assets > Scripts > myCharacter_move.cs > myCharacter_move > moveSpeed
1  using UnityEngine;
2
3  0 references
4  public class myCharacter_move : MonoBehaviour
5  {
6      1 reference
7      [SerializeField] float steerSpeed = 0.5f;
8      1 reference
9      [SerializeField] float moveSpeed = 0.01f;
10     // Start is called once before the first execution of Update after the MonoBehaviour is created
11     0 references
12     void Start()
13     {
14
15     }
16
17     // Update is called once per frame
18     0 references
19     void Update()
20     {
21
22         float steerAmount = -Input.GetAxis("Horizontal");
23         float moveAmount = Input.GetAxis("Vertical")*moveSpeed;
24         transform.Rotate(0, 0, steerSpeed * steerAmount);
25         transform.Translate(0, moveAmount, 0);
26     }
27 }
```

New System만 사용할 경우 수정



Edit
Project Settings
Player
Other Settings
Active Input Handling

컴퓨터 성능에 따라 다른 동작

- 고성능 컴퓨터

- Update 메소드가 더 자주 불리게 된다
- 이는 변화가 더 많이 일어나서 캐릭터가 더 빨리 움직이게 됨

- 이를 해결하는 방법

- 고성능 컴퓨터에서는 Update 안에서 조금만 움직이고
- 저성능 컴퓨터에서는 Update 안에서 더 많이 움직인다

- 어떻게 이를 구현할 것인가?

- 이전 Update에서 이번 Update가 불리는 데에 걸린 시간
 - Time.deltaTime
 - 고성능 컴퓨터: 작은 deltaTime
 - 저성능 컴퓨터: 큰 deltaTime

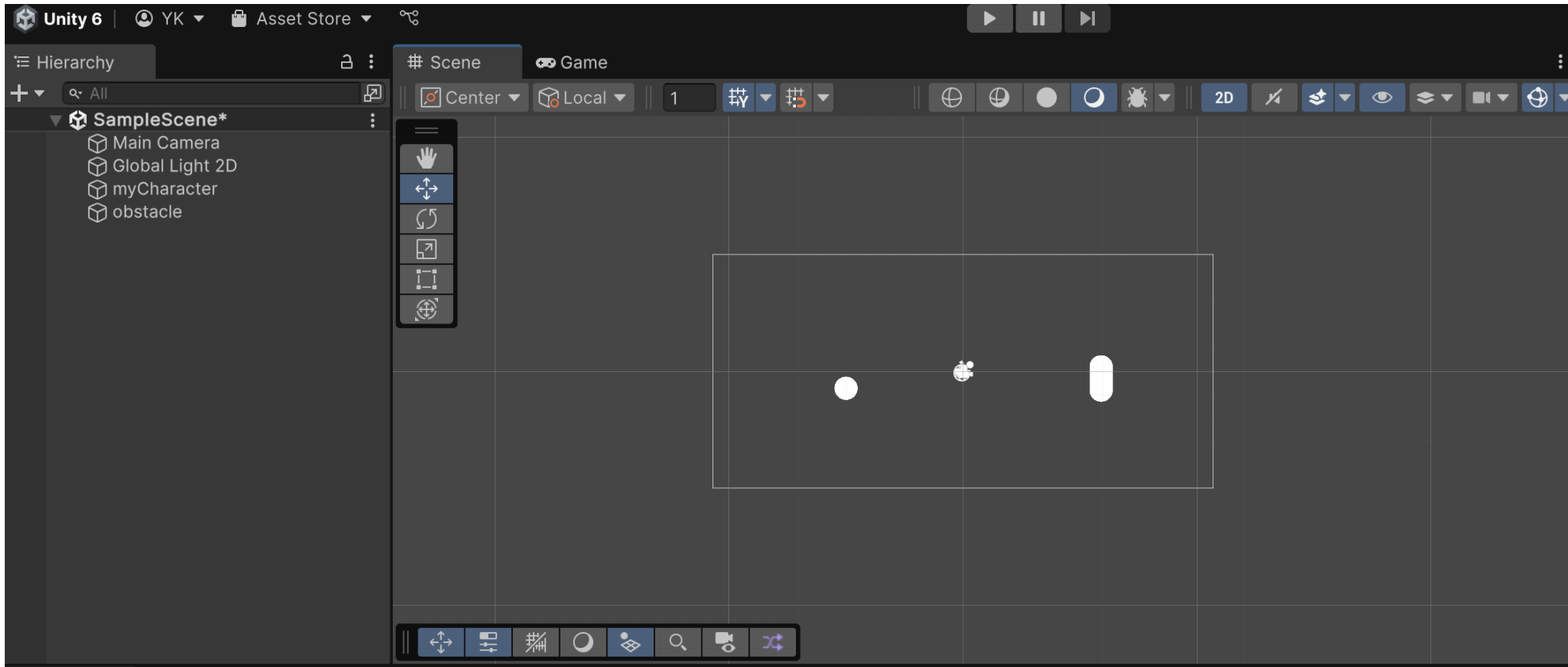


deltaTime을 변화의 규모에 곱하면!!

컴퓨터 성능에 따라 다른 동작

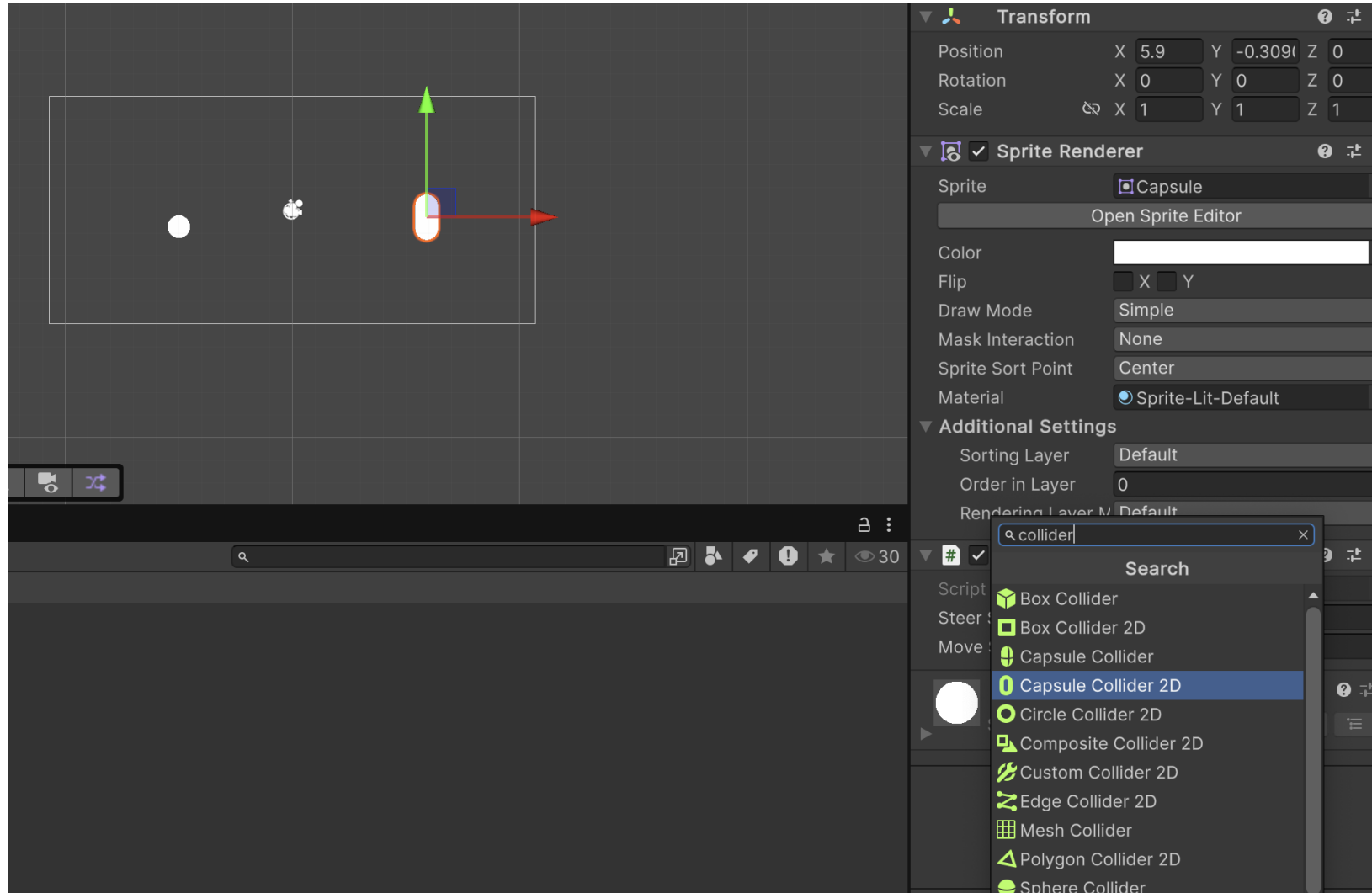
```
1  using UnityEngine;
2
3  0 references
4  public class myCharacter_move : MonoBehaviour
5  {
6      1 reference
7      [SerializeField] float steerSpeed = 100f;
8      1 reference
9      [SerializeField] float moveSpeed = 10f;
10     // Start is called once before the first execution of Update after the MonoBehaviour is created
11     0 references
12     void Start()
13     {
14
15     }
16
17     // Update is called once per frame
18     0 references
19     void Update()
20     {
21         float steerAmount = -Input.GetAxis("Horizontal") * Time.deltaTime;
22         float moveAmount = Input.GetAxis("Vertical") * moveSpeed * Time.deltaTime;
23         transform.Rotate(0, 0, steerSpeed * steerAmount);
24         transform.Translate(0, moveAmount, 0);
25     }
26 }
```

Collider



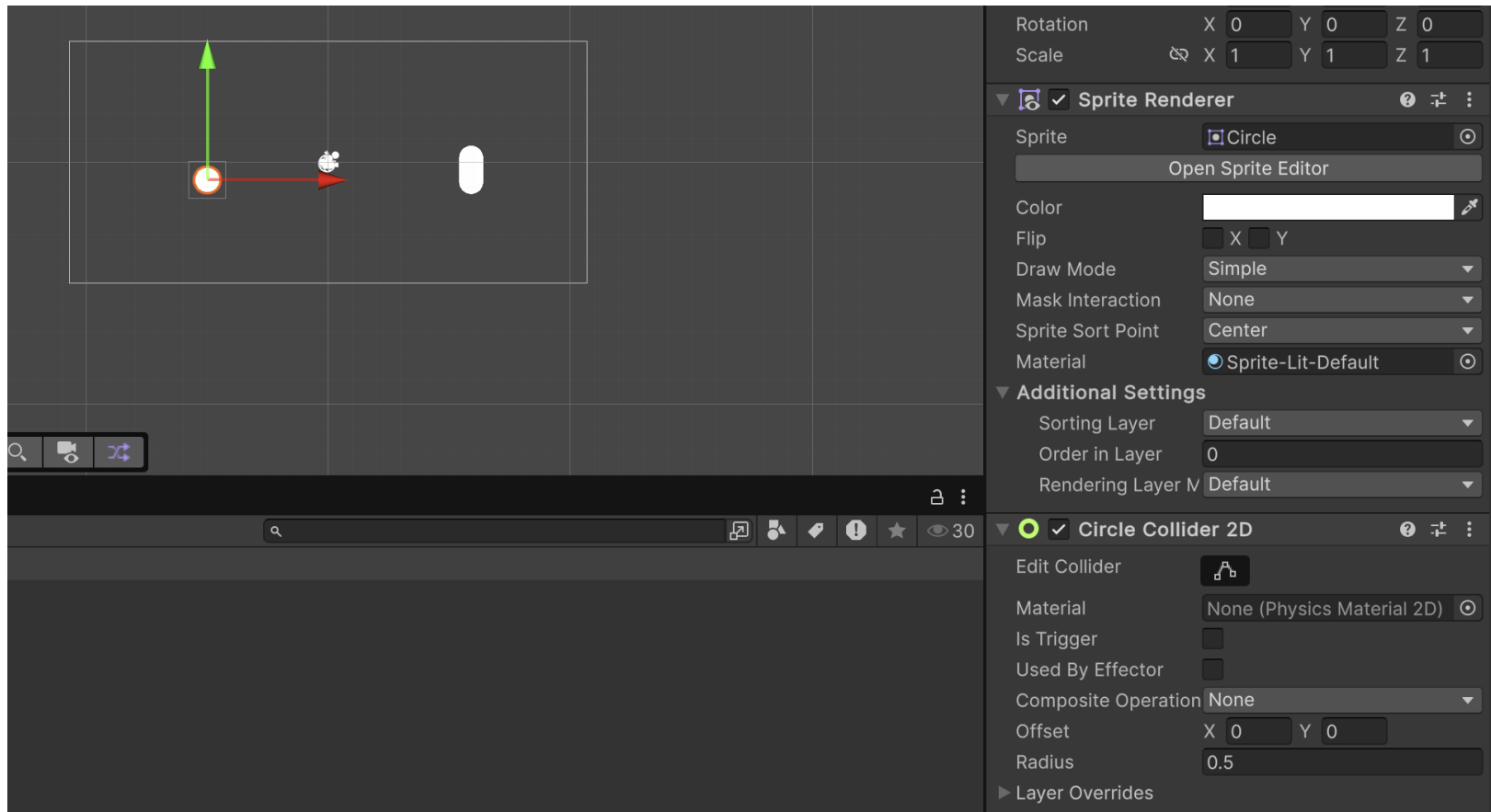
새로운 객체 추가 (Circle)

각각의 객체에 Collider 컴포넌트 추가



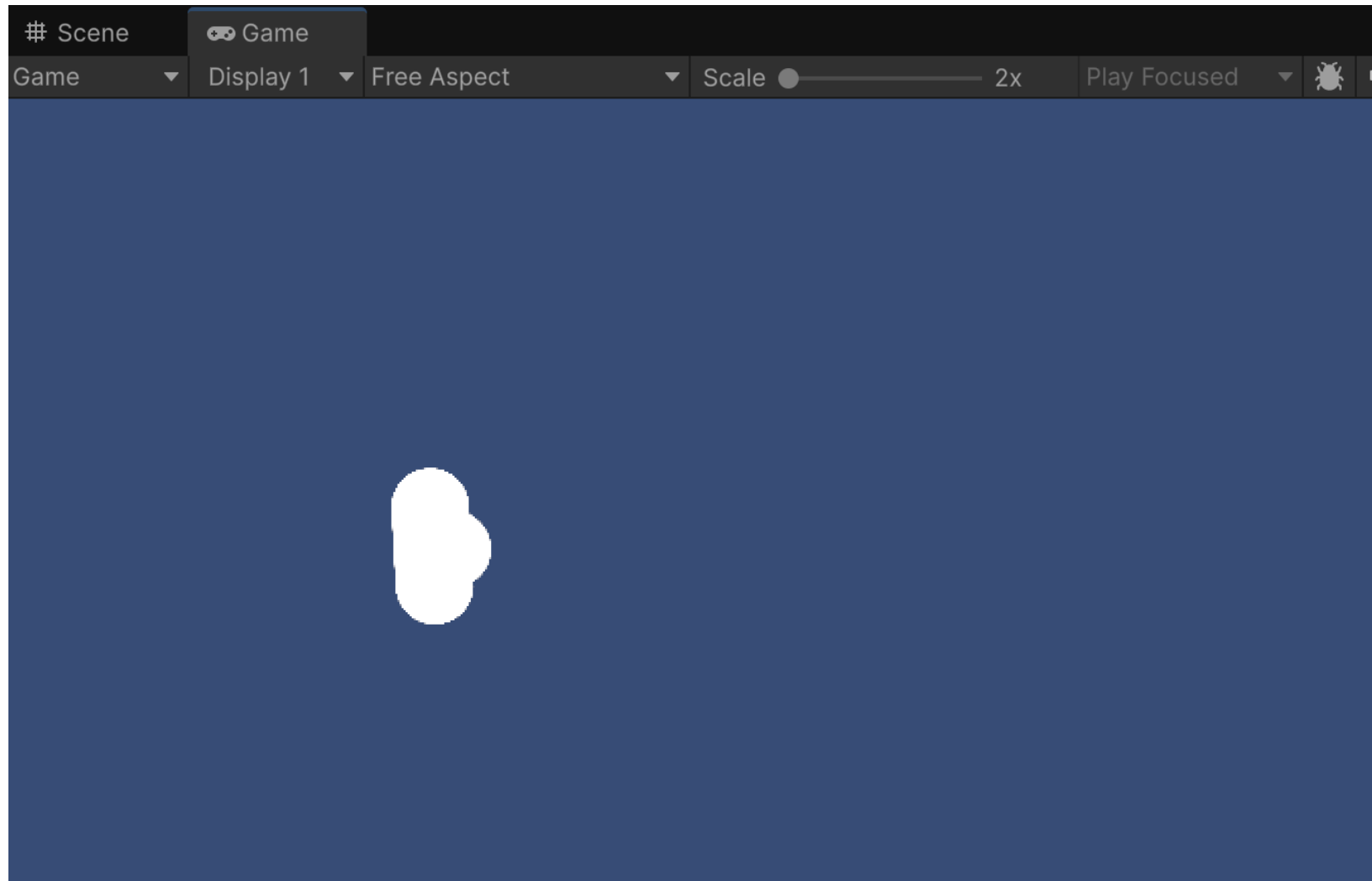
객체 선택
Add Component
Capsule Collider 2D

각각의 객체에 Collider 컴포넌트 추가



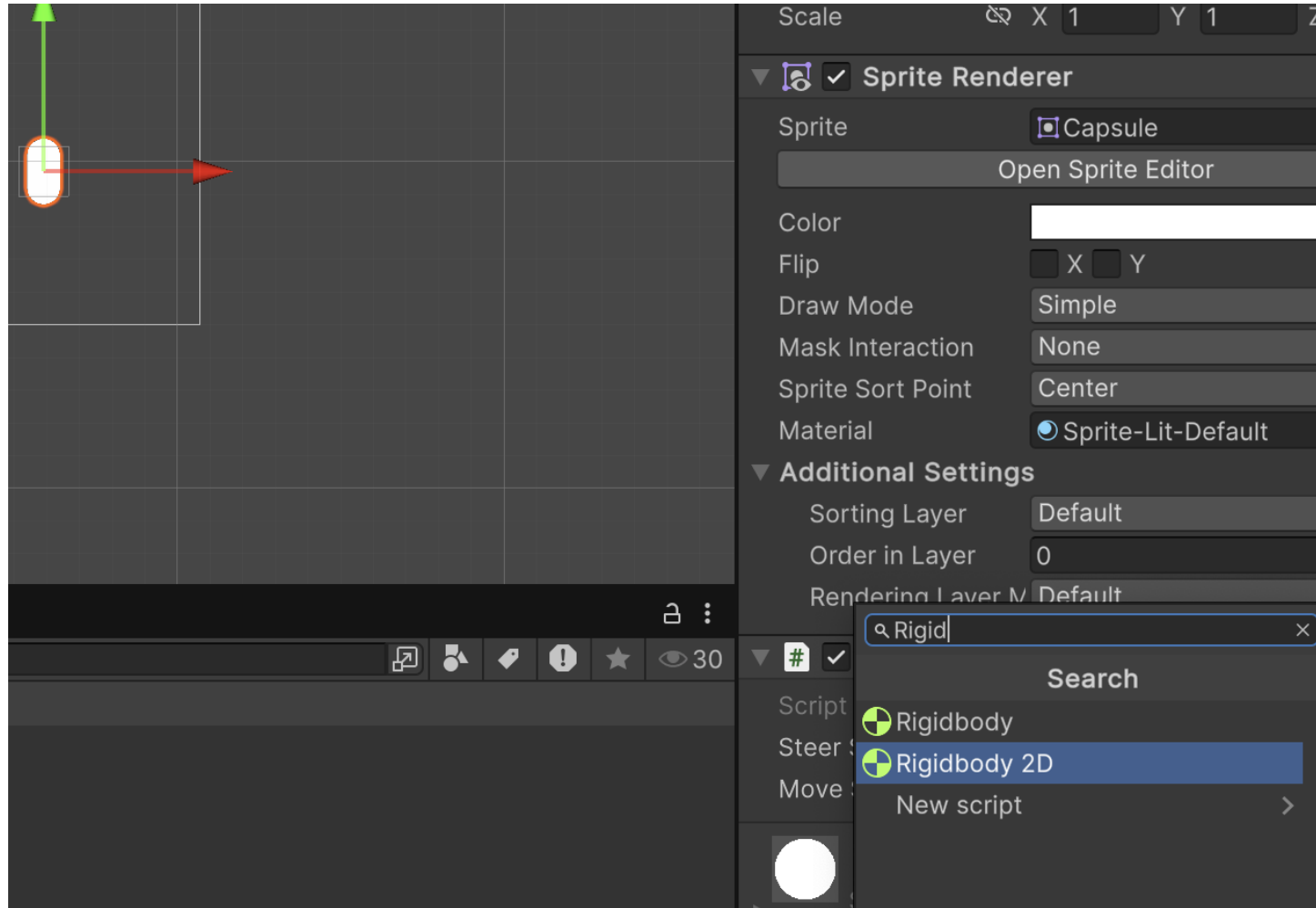
원 객체 선택
Add Component
Circle Collider 2D

충돌 처리가 안 됨



두 객체 겹쳐서 나타남

물리 시스템을 추가

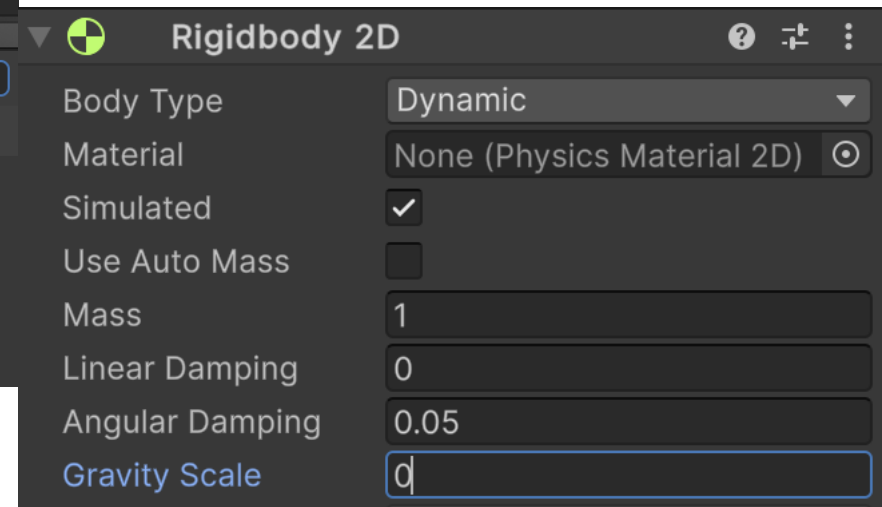


Rigidbody 2D Component

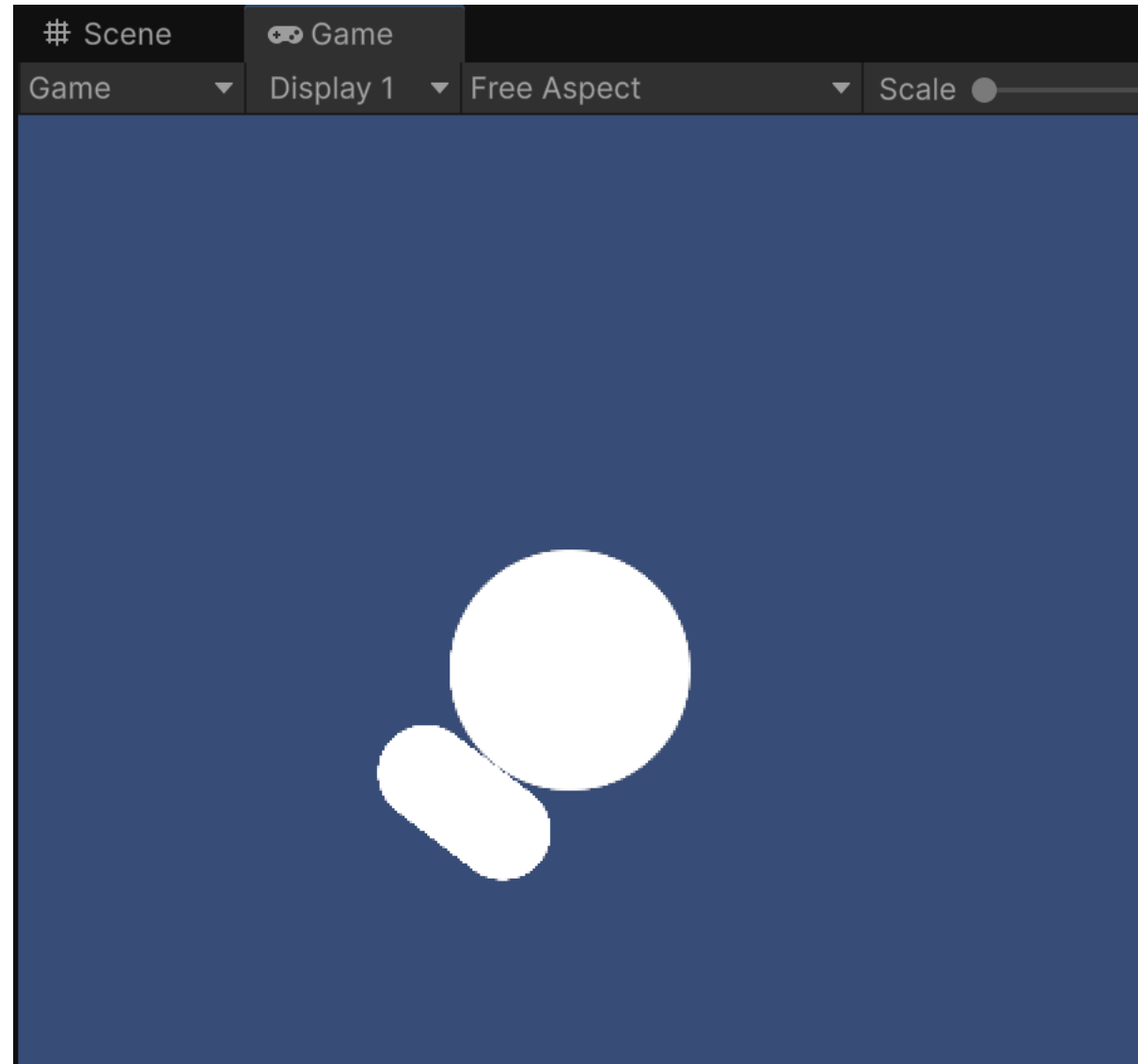
- 충돌이 발생했을 때의 처리
- 강제운동 시뮬레이션으로 처리

중력을 받아 떨어질 것

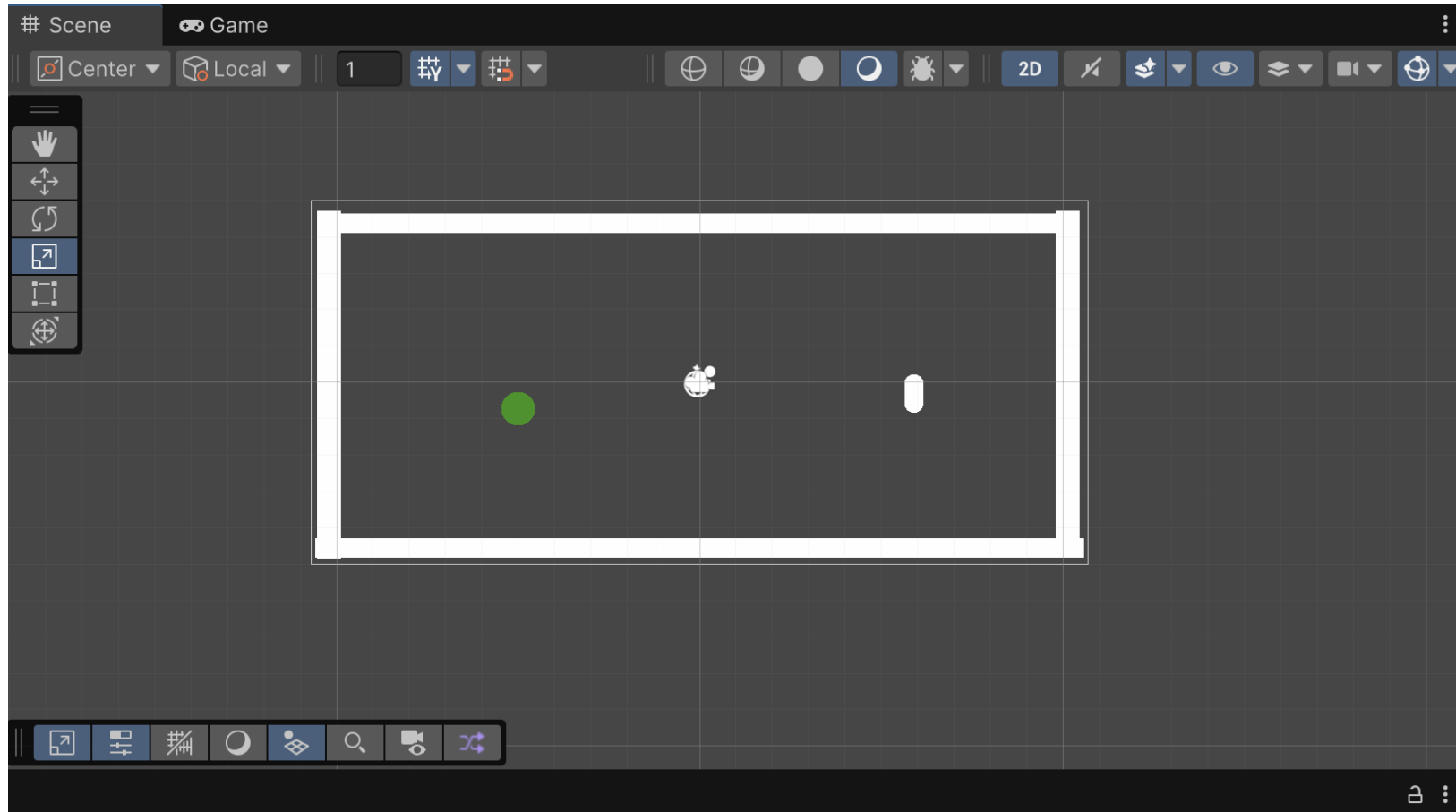
- 중력을 제거하자



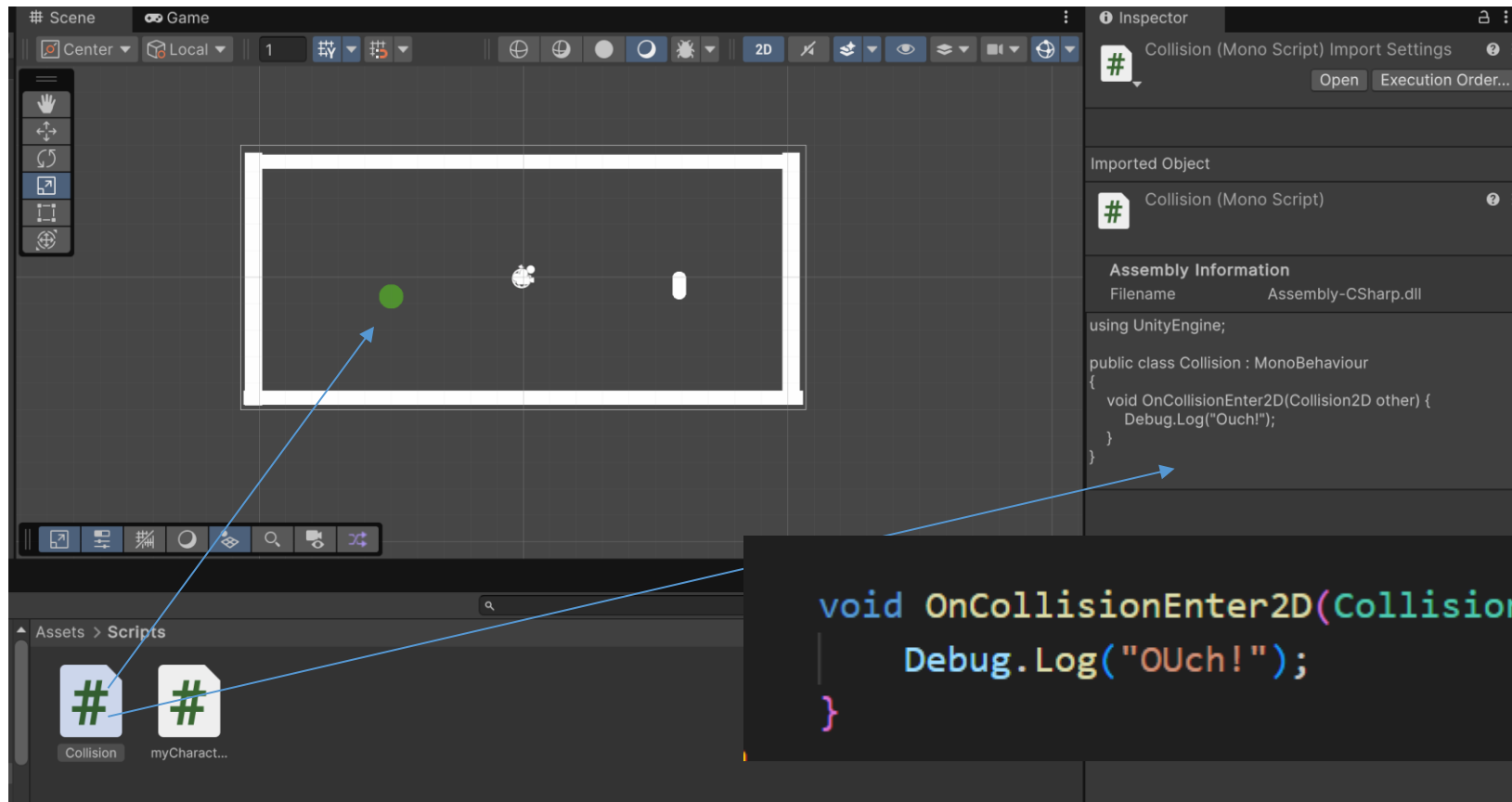
물리 시스템을 추가



벽을 추가해 보라



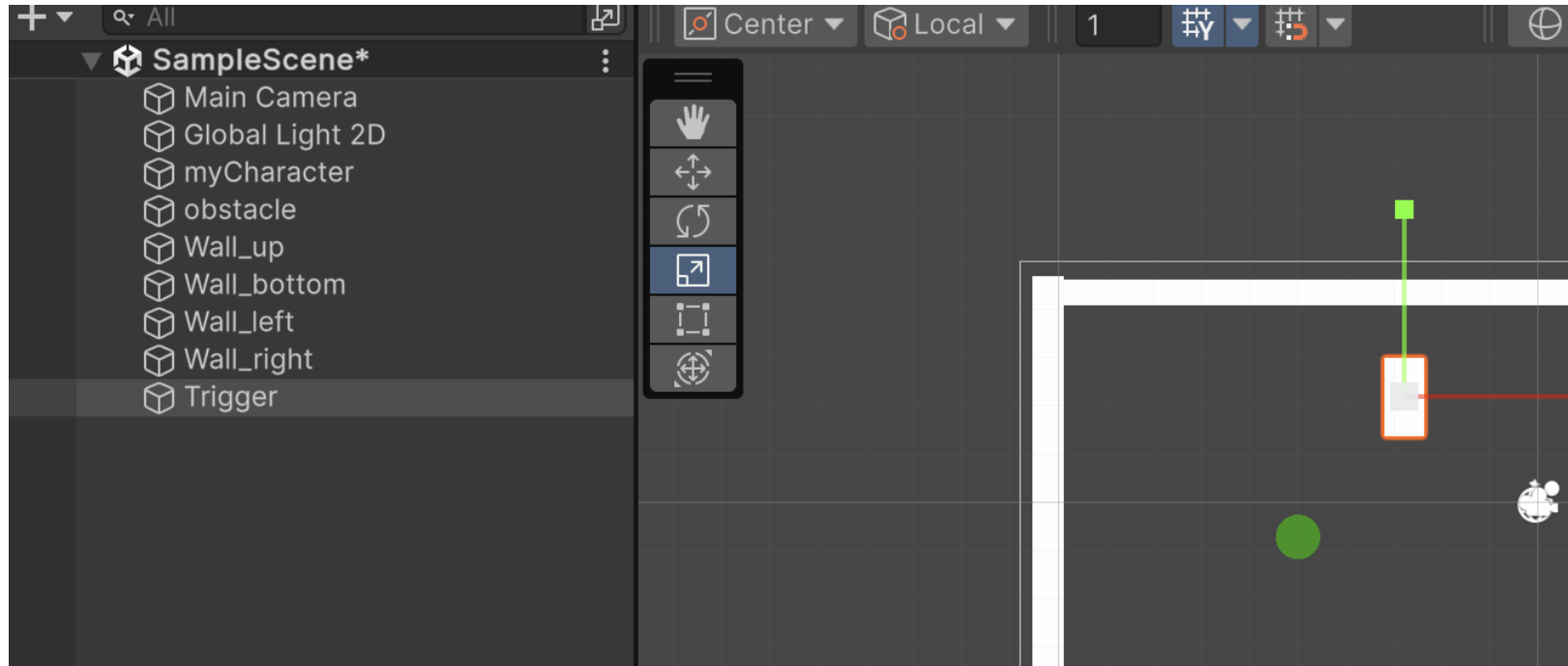
충돌시에 특정한 동작을 하게 만들자



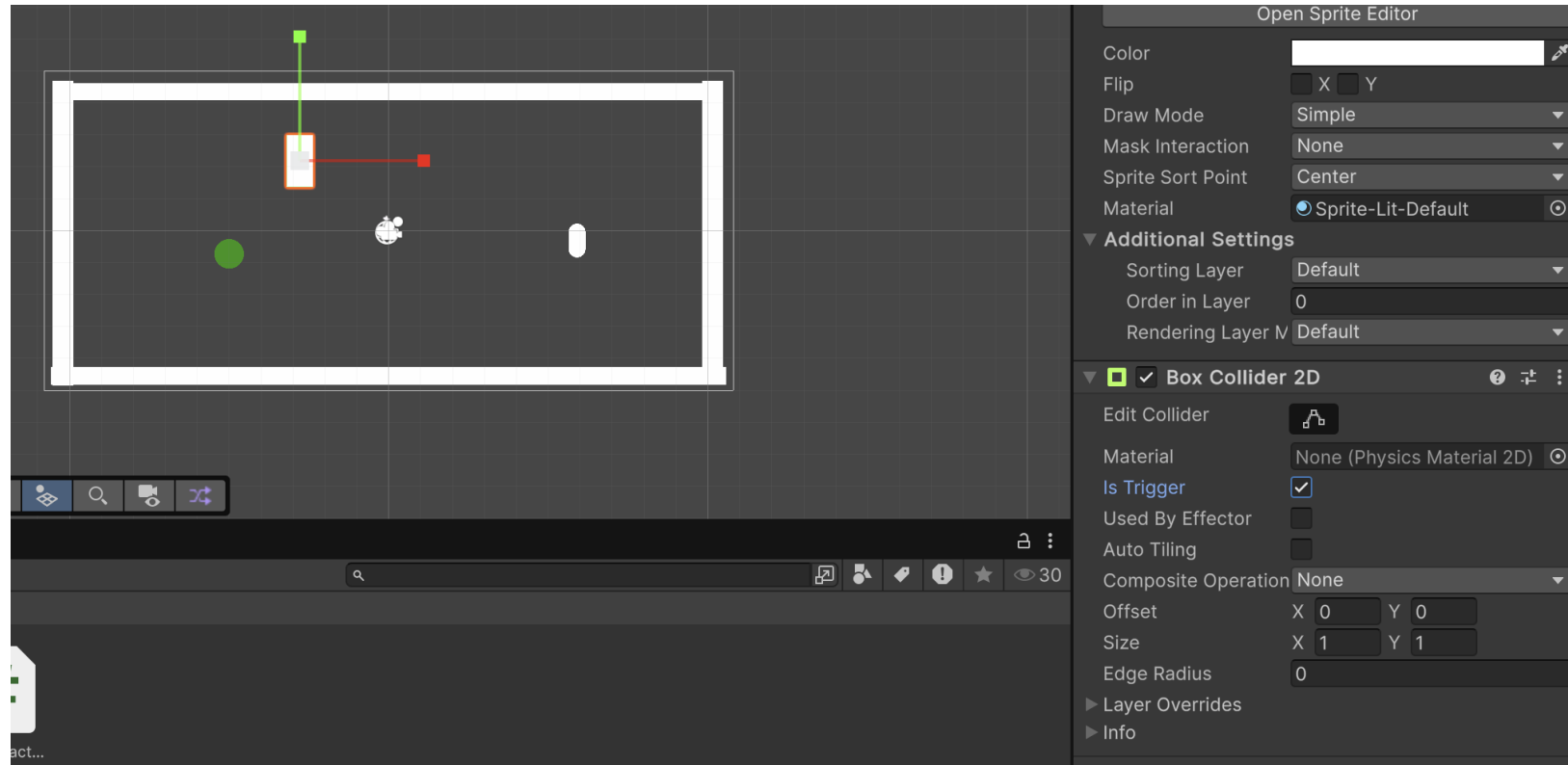
Trigger – Collider의 차이

- Collider
 - 물리적인 충돌 반응이 나타남
 - Rigidbody인 경우
 - 물체가 강체의 특성에 따라 움직임
 - Rigidbody가 아닌 경우
 - Static한 물체로 제자리에 머물며 상대를 밀어냄
- Trigger
 - 물리적인 충돌 반응이 없음
 - 단순히 두 물체가 겹쳤음을 감지
 - 감지 이후의 동작은 OnTriggerEnter2D로 처리

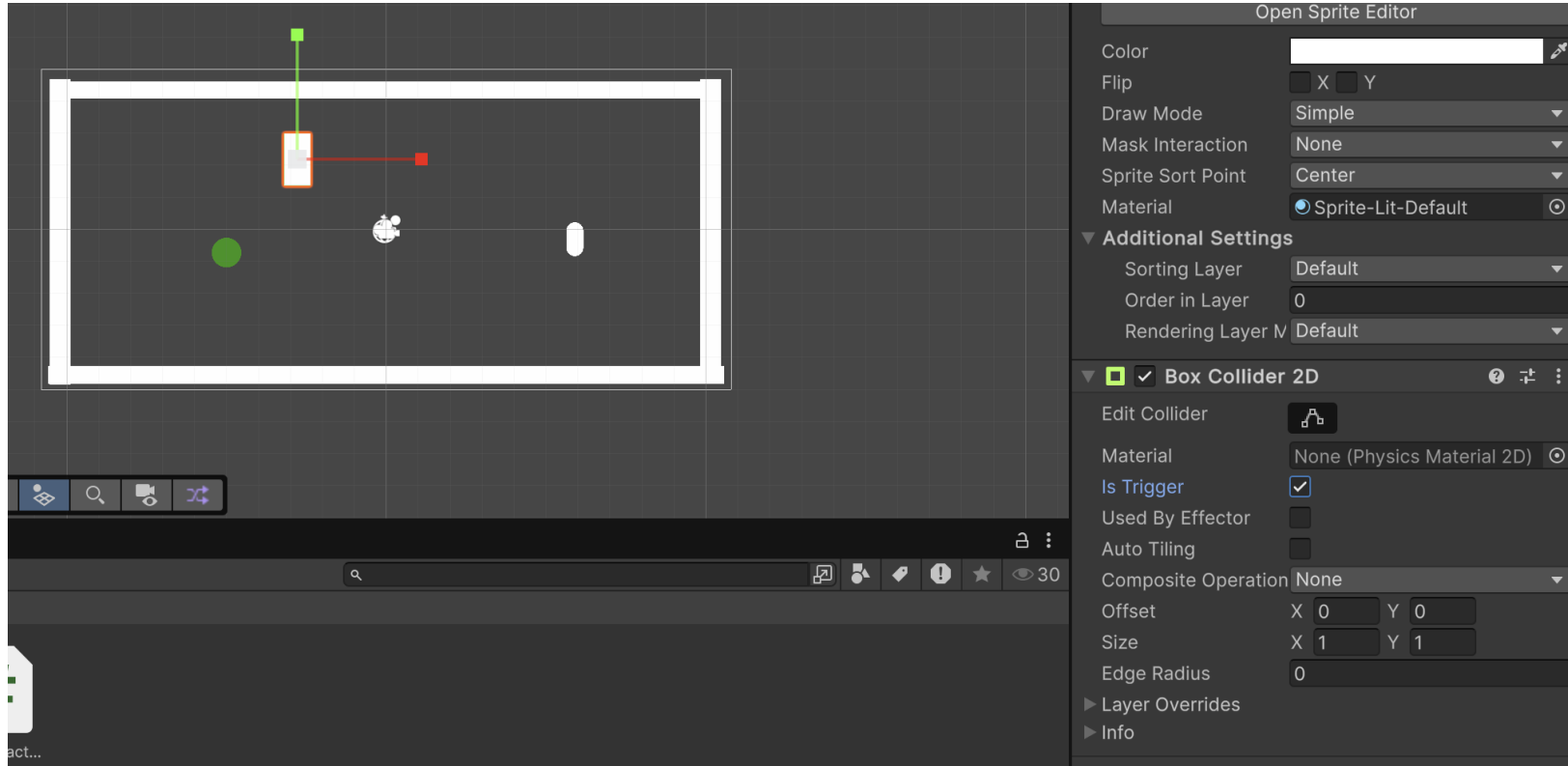
Trigger – Collider의 차이



Trigger – Collider를 추가한 뒤에 Trigger 선택



Collision 스크립트를 새 트리거에도 연결해 보자



캐릭터를 트리거에 충돌시키자

- 아무런 반응 없음
- Collision Script의 OnCollisionEnter2D도 동작하지 않음

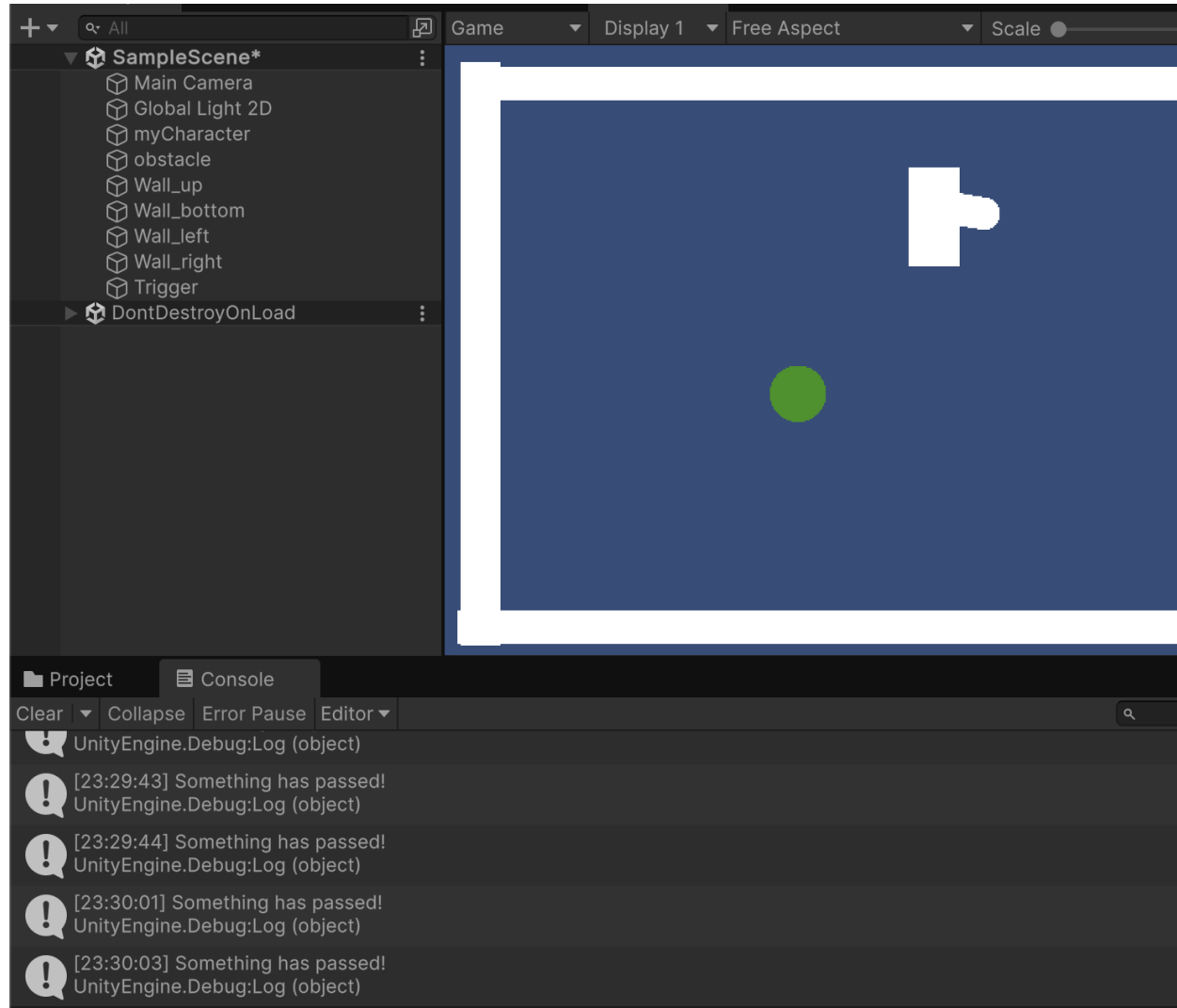
무엇을 할 수 있나?

- OnTriggerEnter2D 동작

OnTriggerEnter2D

```
C# Collision.cs ×
Assets > Scripts > C# Collision.cs > Collision > OnTriggerEnter2D
1
2 using UnityEngine;
3
4 0 references
5 public class Collision : MonoBehaviour
6 {
7     0 references
8     void OnCollisionEnter2D(Collision2D other) {
9         Debug.Log("Ouch!");
10    }
11
12     0 references
13     private void OnTriggerEnter2D(Collider2D other){
14         Debug.Log("Something has passed!");
15     }
16 }
```

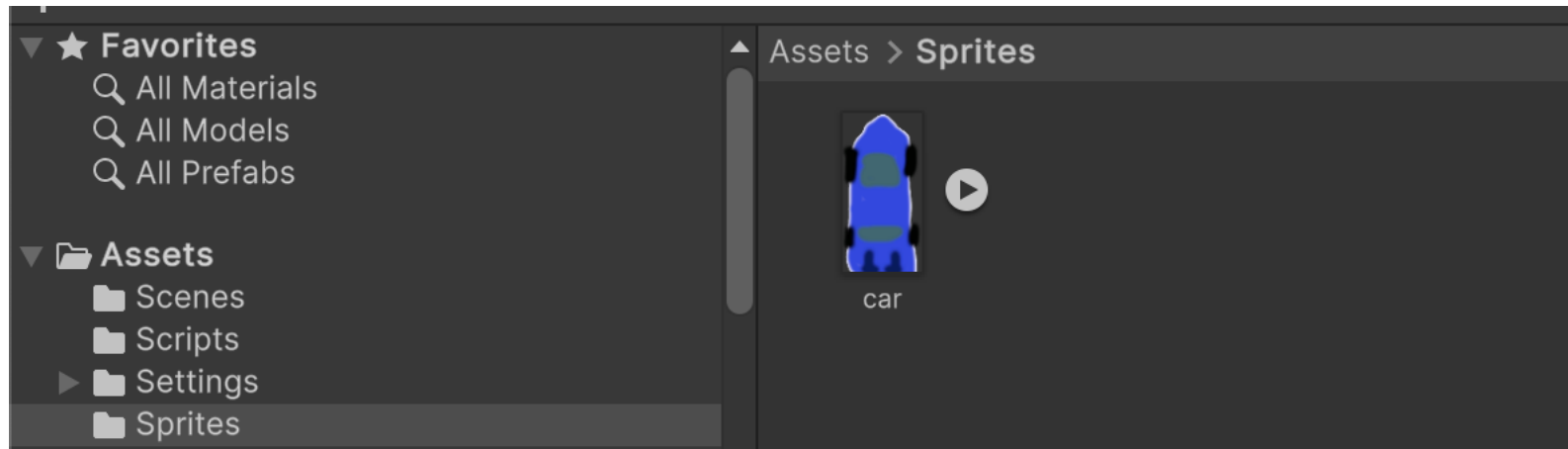
OnTriggerEnter2D



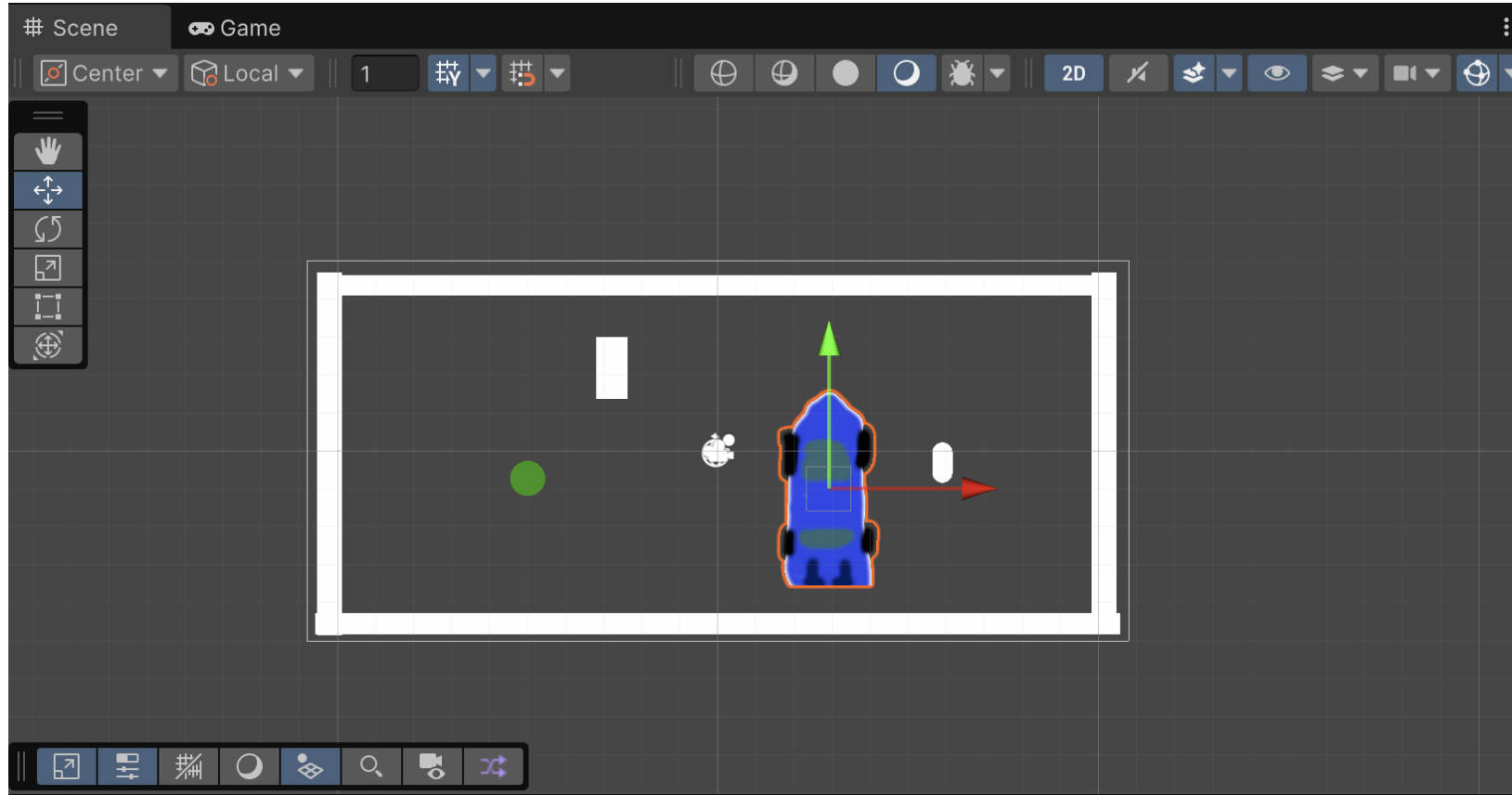
다양한 활용 방법을 생각해 보자

Asset

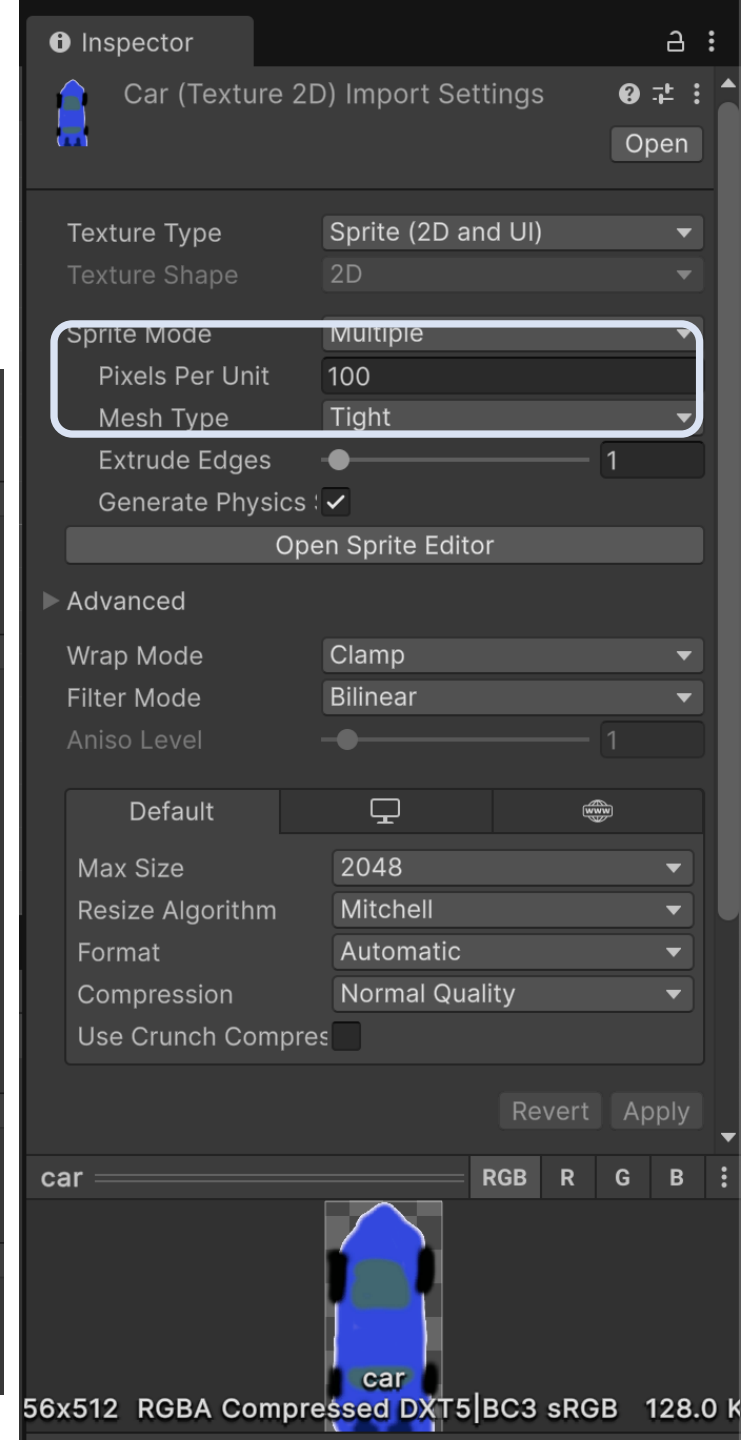
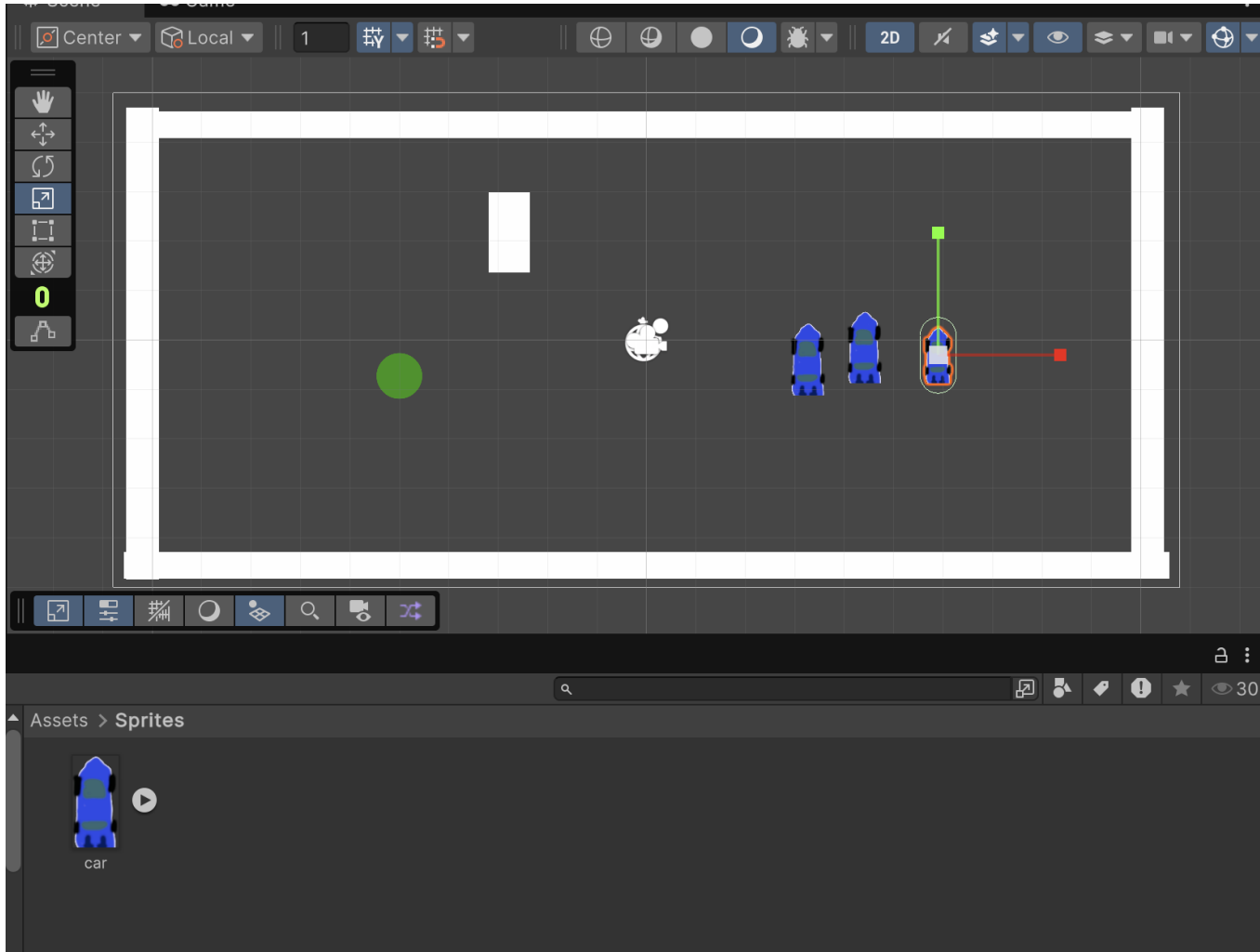
Assets 폴더 아래 자유롭게



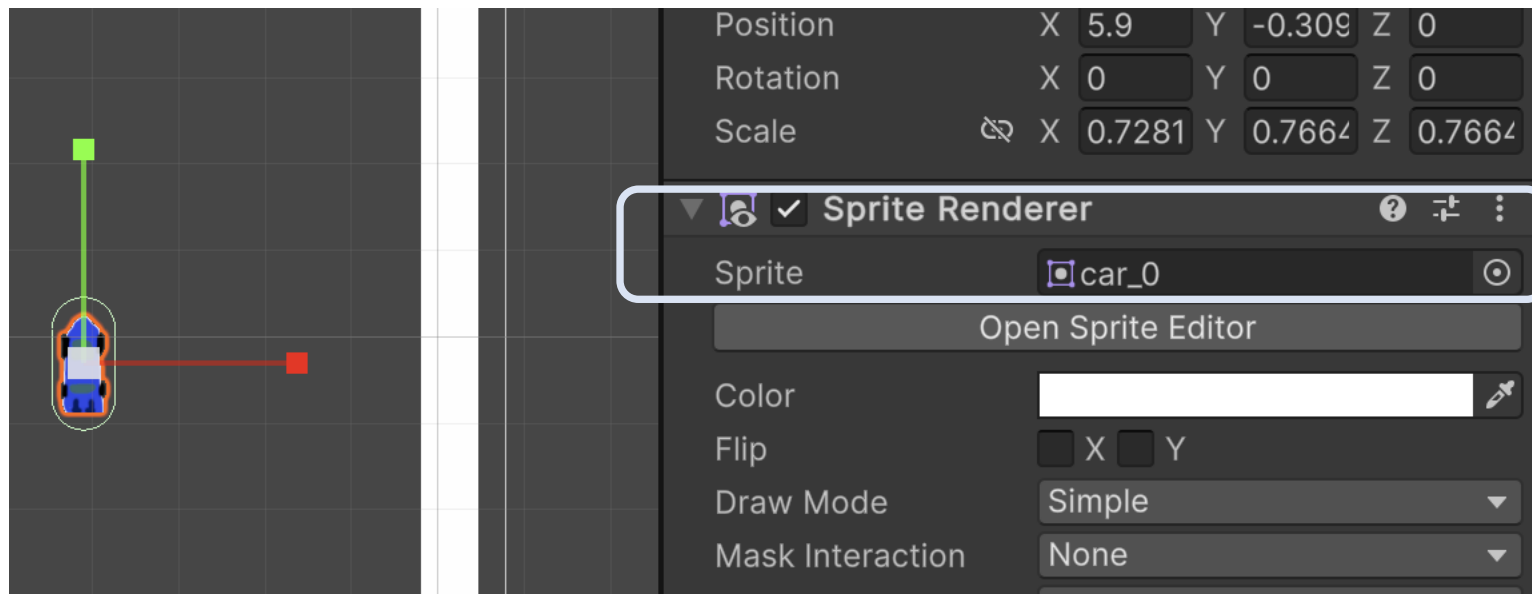
Asset – 크기 문제 해결 필요



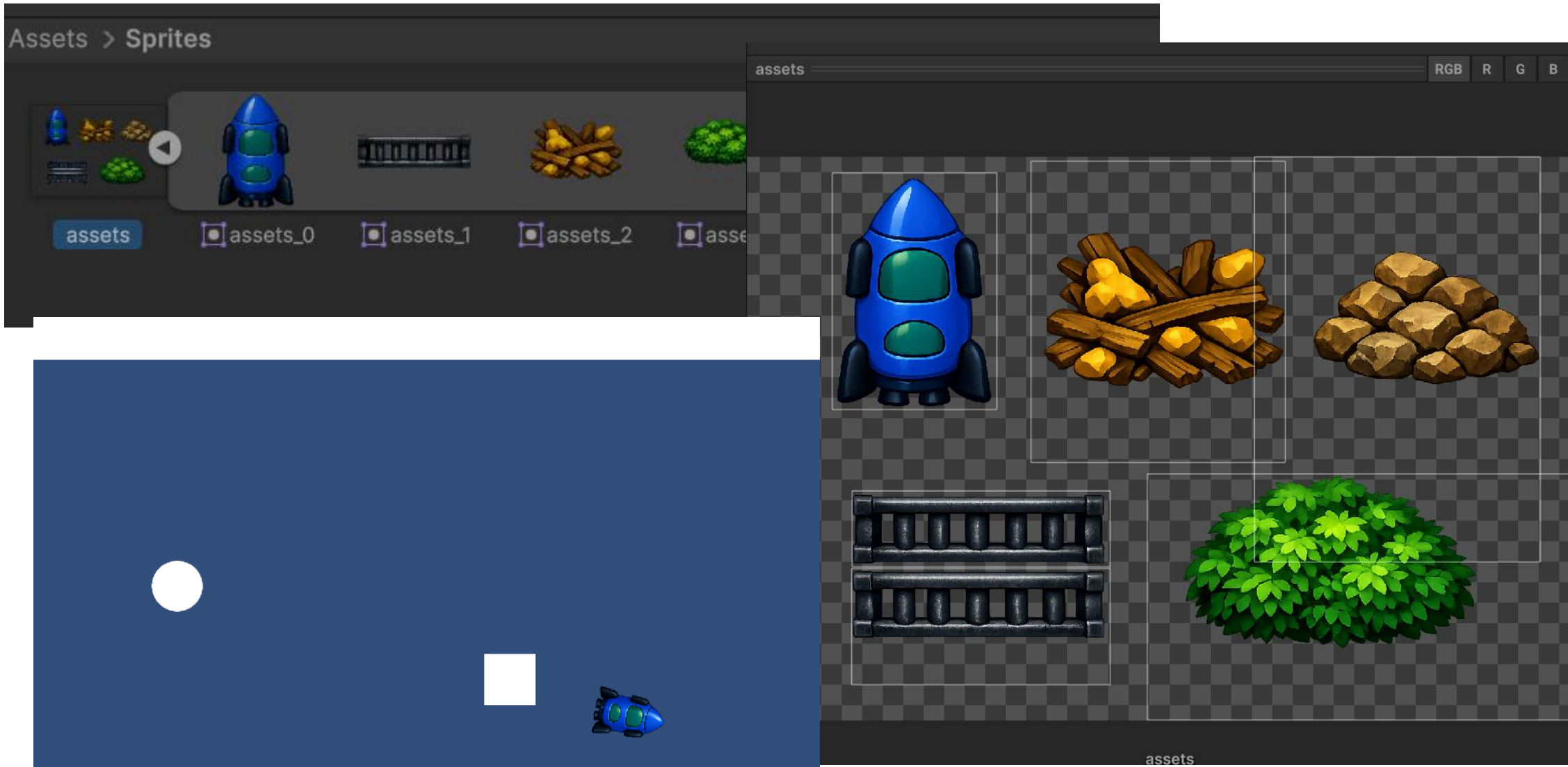
Asset - 크기 문제 해결 필요



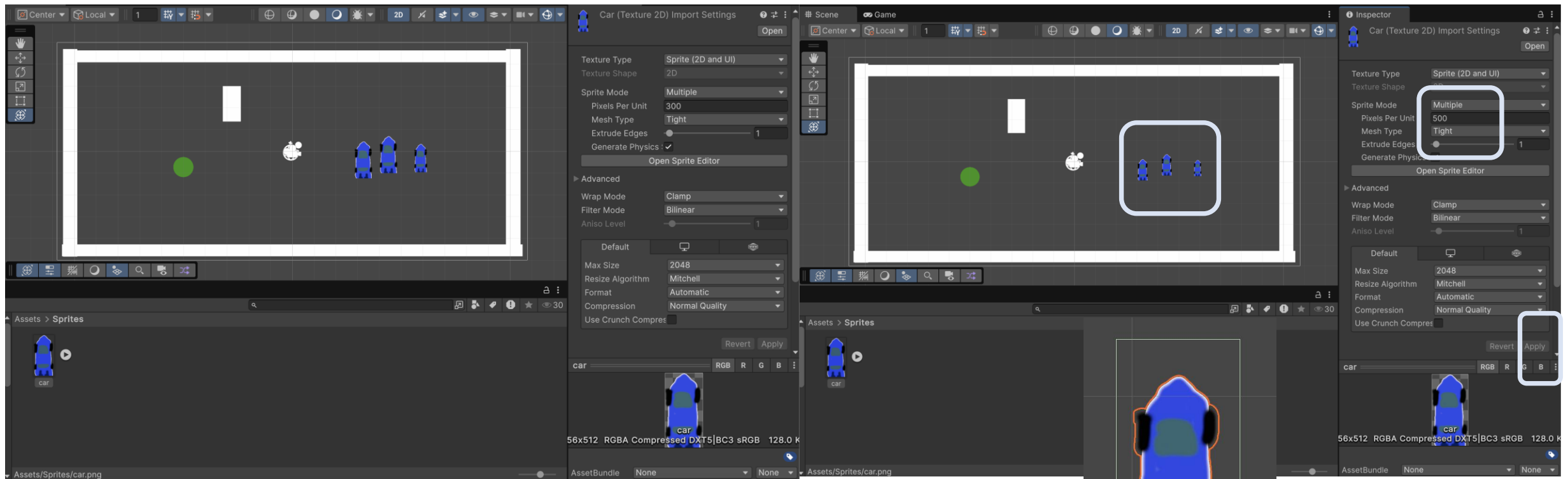
차 캐릭터에 적용



스프라이트 시트 사용해 보기



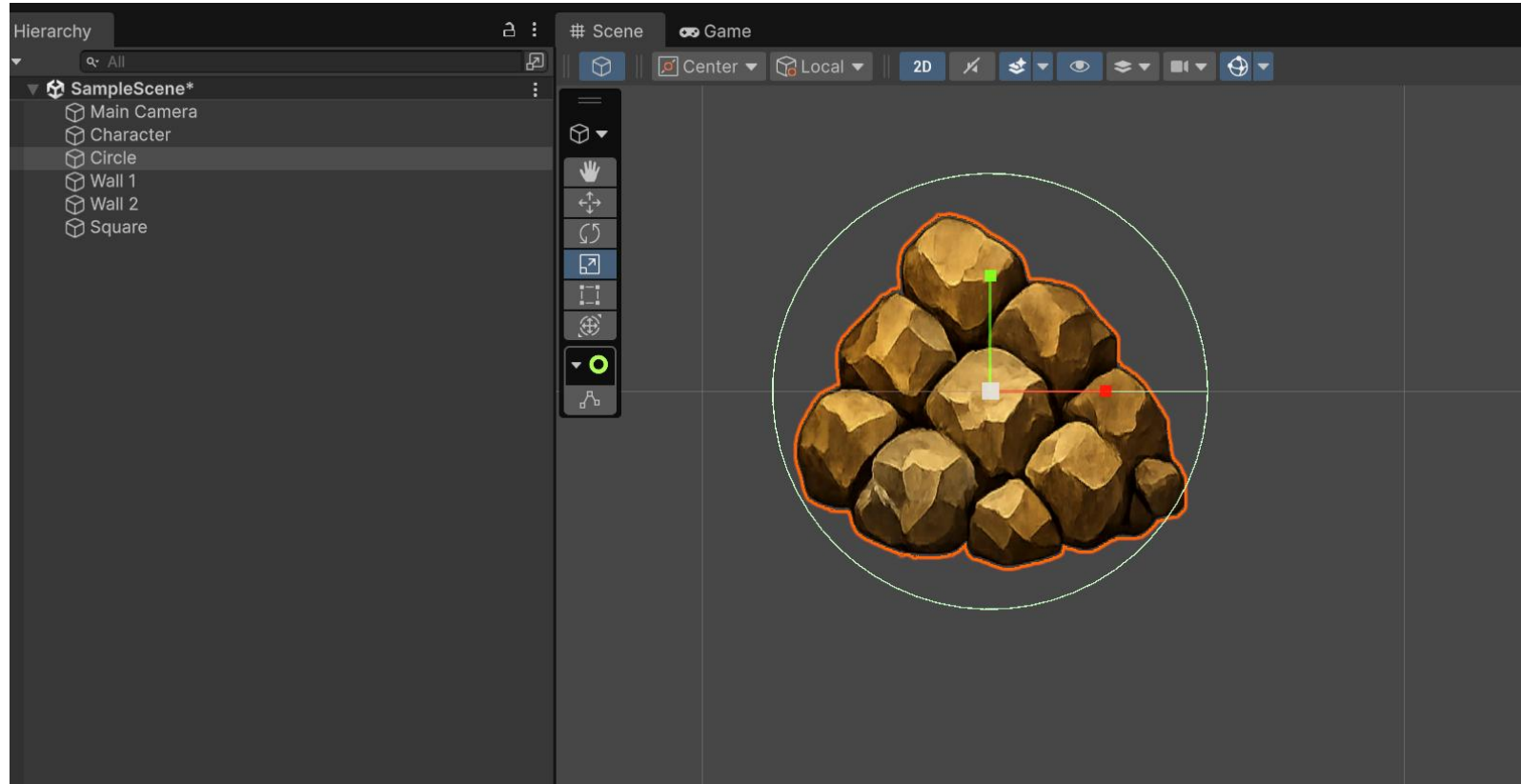
Asset을 선택하여 inspector로 고치면 다른 방식으로 적용



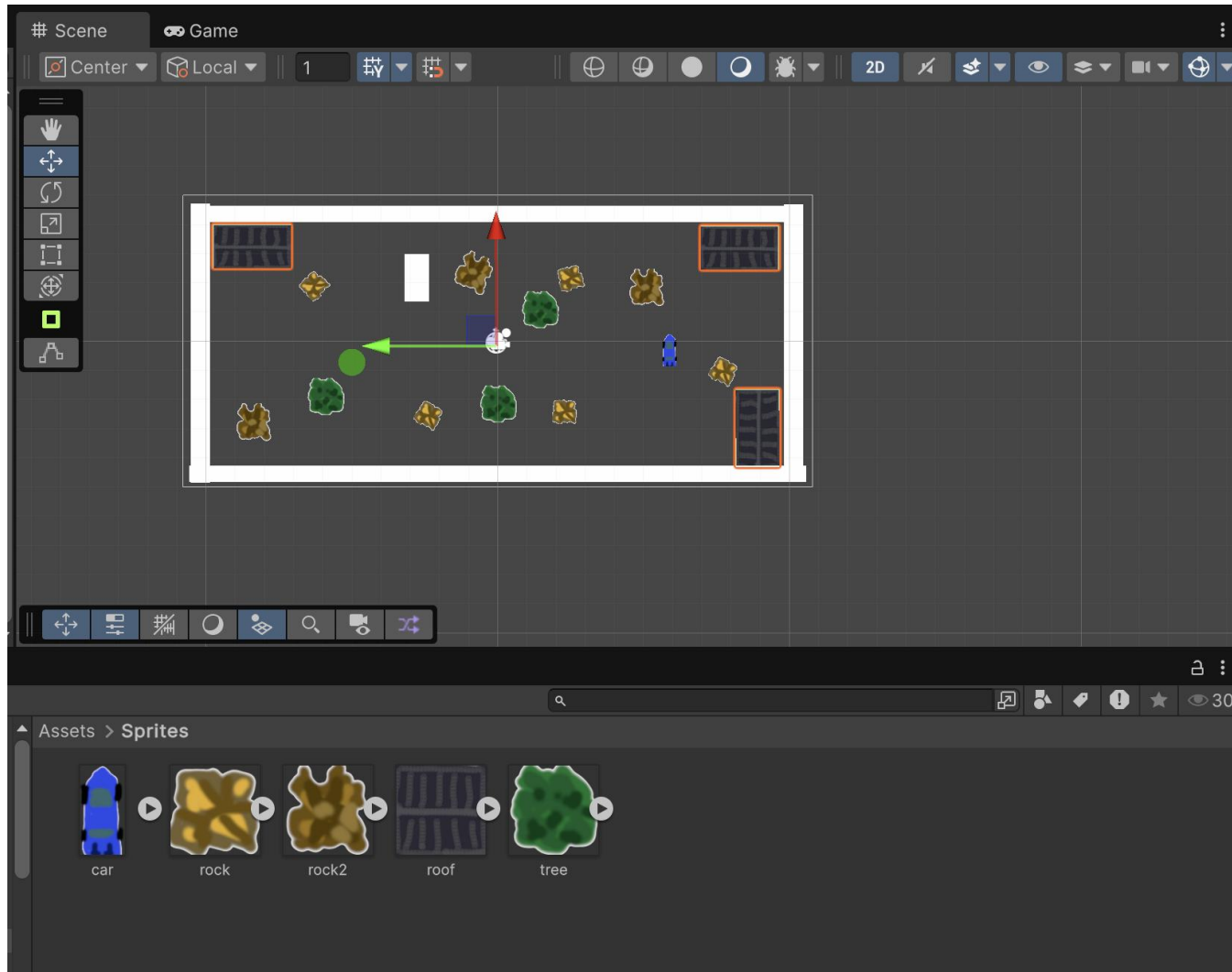
적용된 모든 스프라이트가 변화함

콜라이더와 비교하면
작게 입혀진 것을 확인

스프라이트 시트의 경우 캐릭터를 변형하는 방식 가능



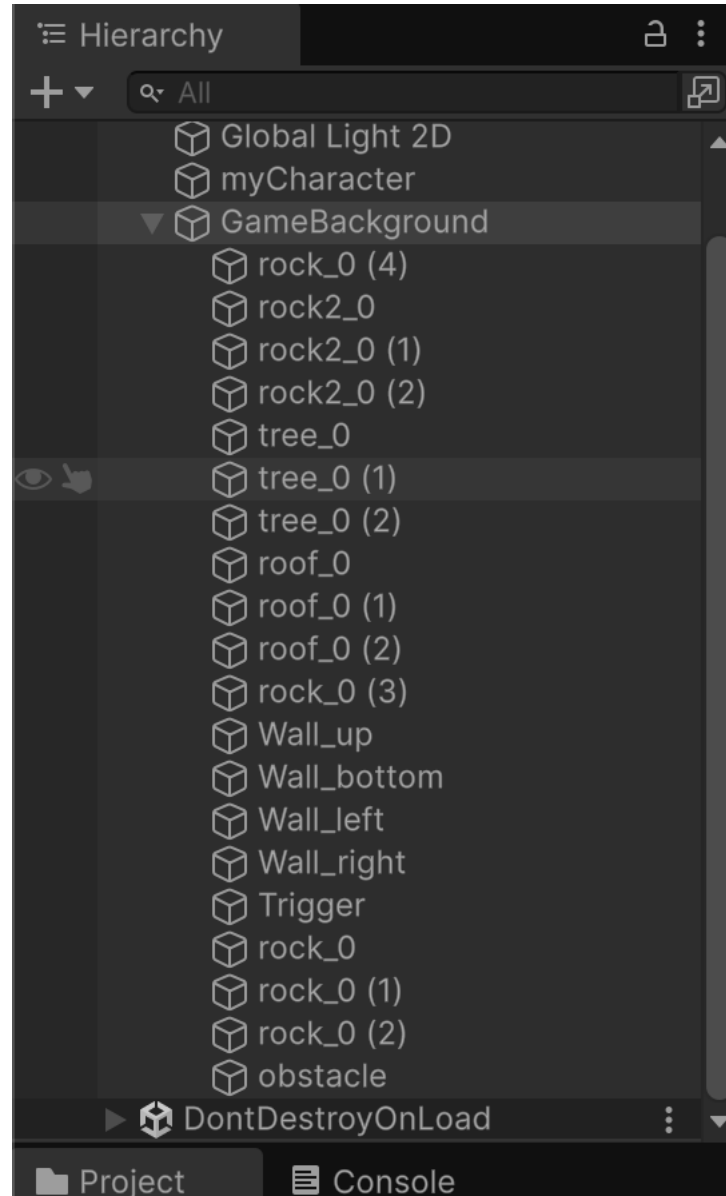
다양한 물체를 배치하여 장면을 구성해 보자



나무와 집은 충돌시 움직이지 않고,
돌은 움직이게 만들어 보라.

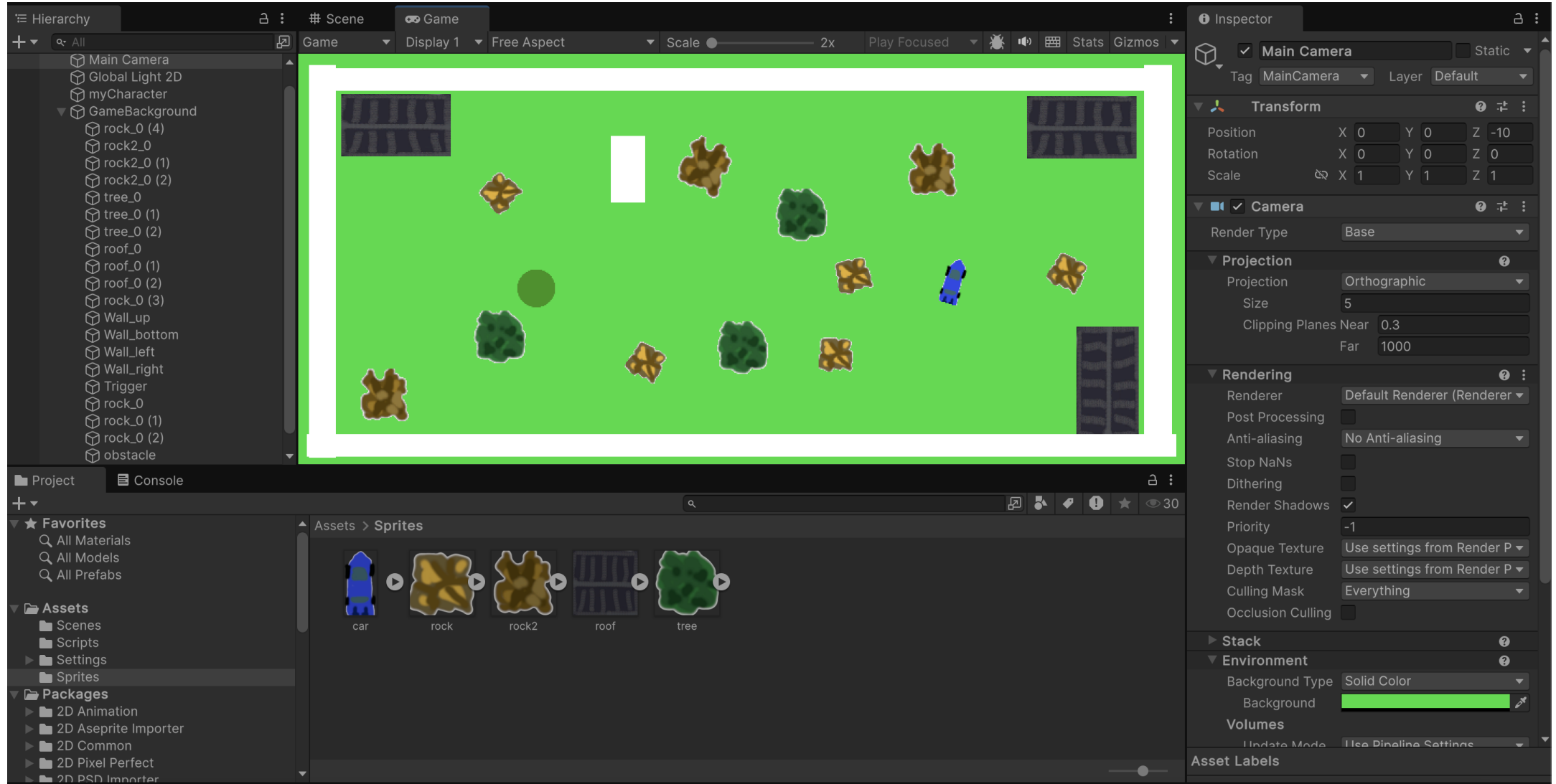
객체의 계층 관리

빈 객체를 만들어 그 아래에 담는다



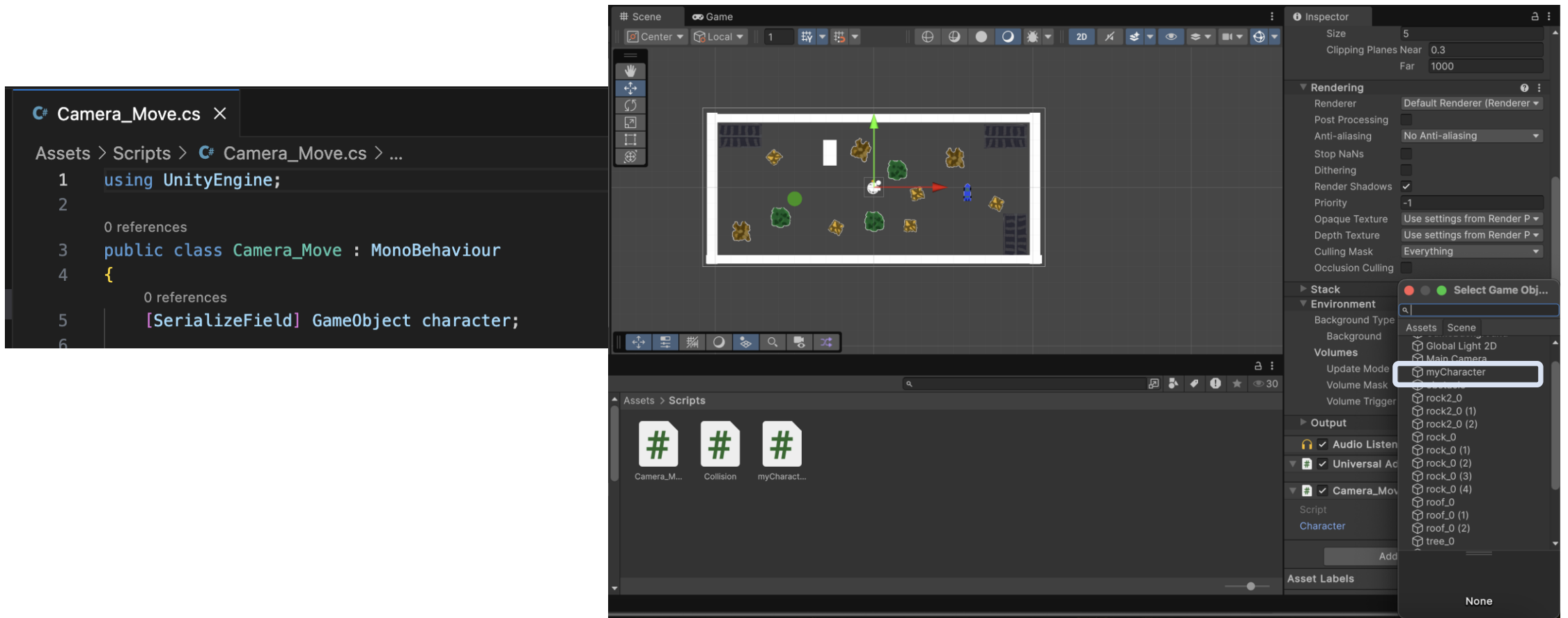
카메라 관리

카메라를 선택하여 배경색 바꿔 보자



카메라가 나의 캐릭터를 따라 움직이게 해 보자

카메라를 선택하여 Camera_Move.cs 스크립트를 연결해 보자.



카메라가 나의 캐릭터를 따라 움직이게 해 보자

```
// Update is called once per frame
```

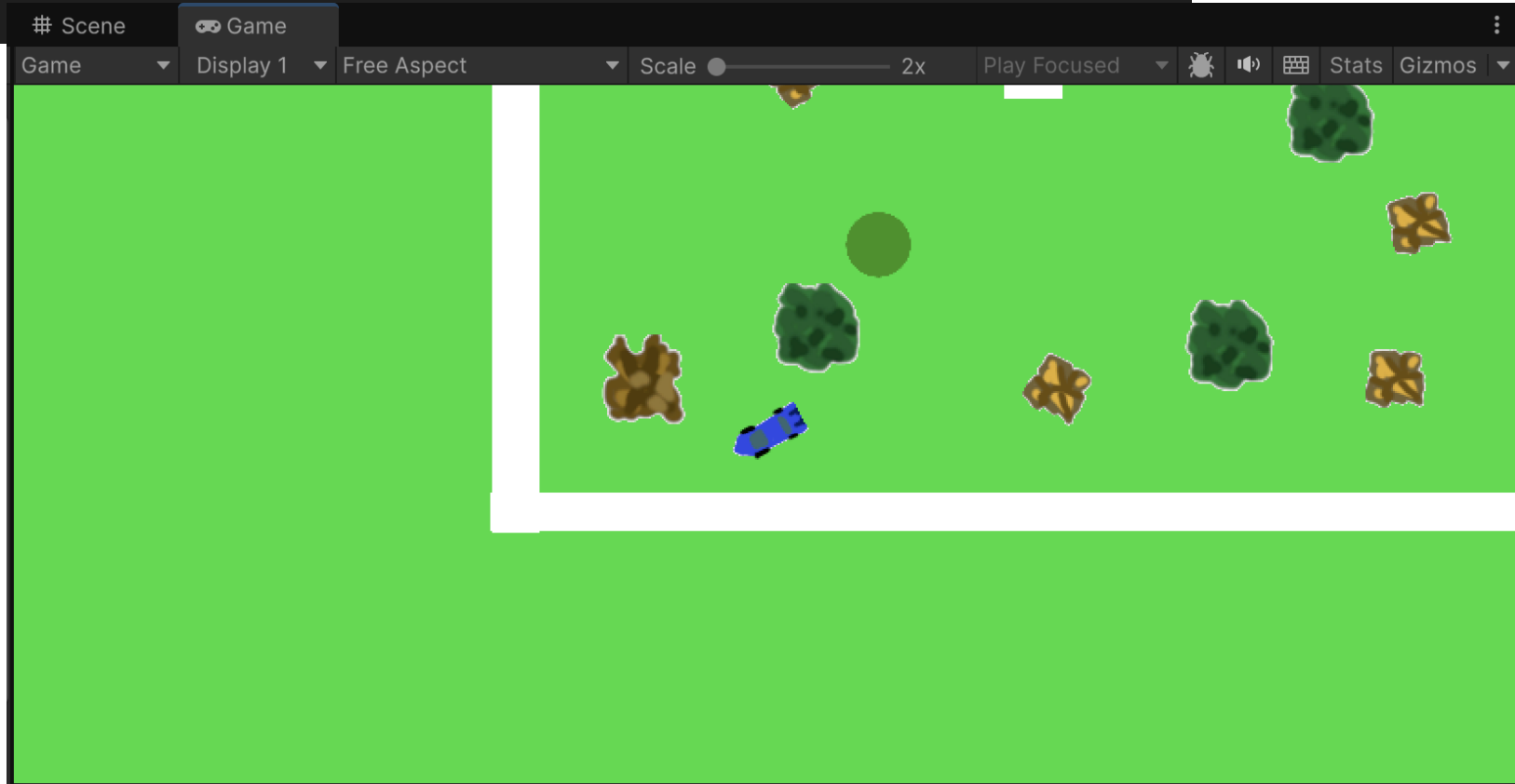
```
0 references
```

```
void Update()
```

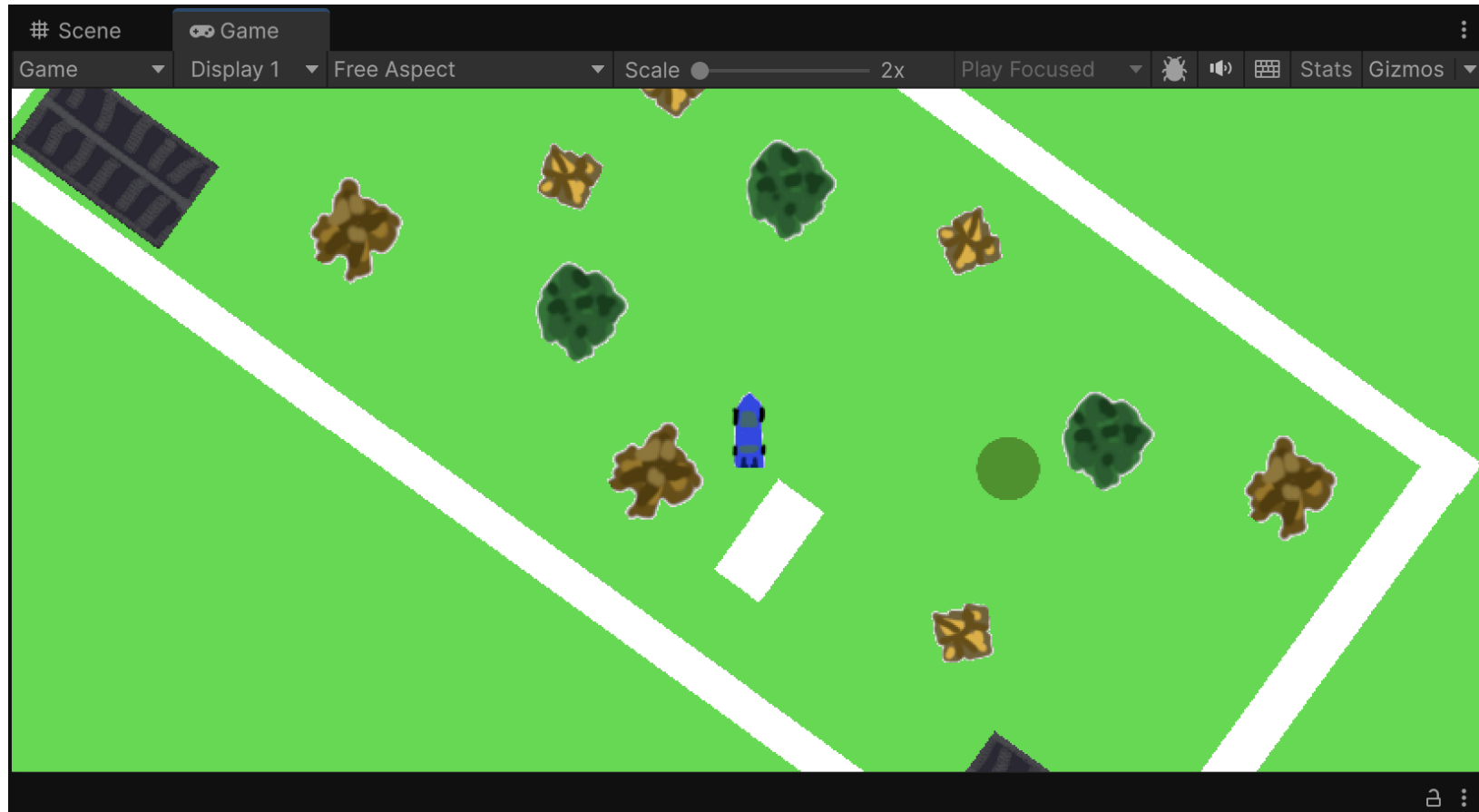
```
{
```

```
    transform.position = character.transform.position + new Vector3(0, 0, -10);
```

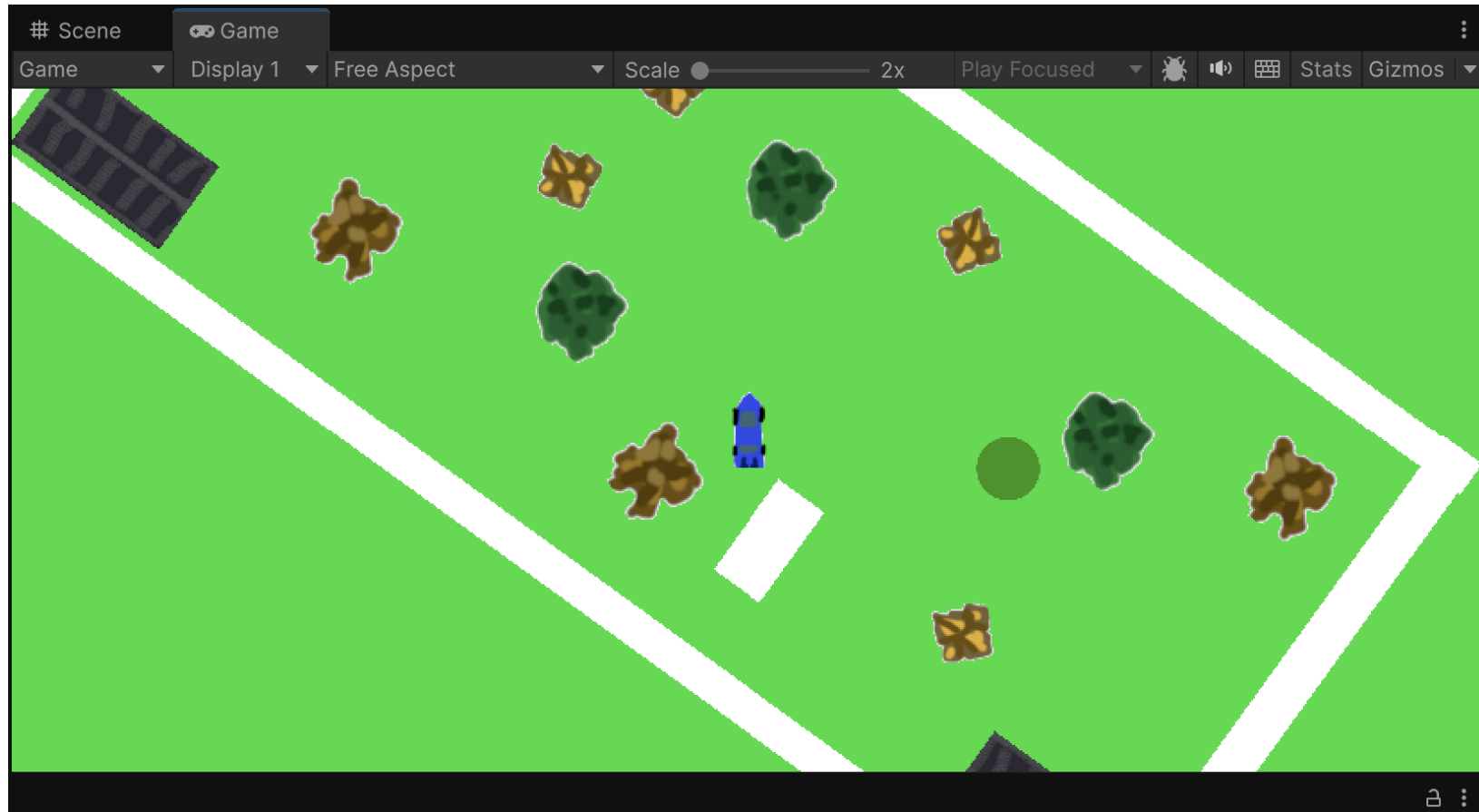
```
}
```



이렇게 만드려면 어떻게 해야 할까?



이렇게 만드려면 어떻게 해야 할까?



If 문과 tag

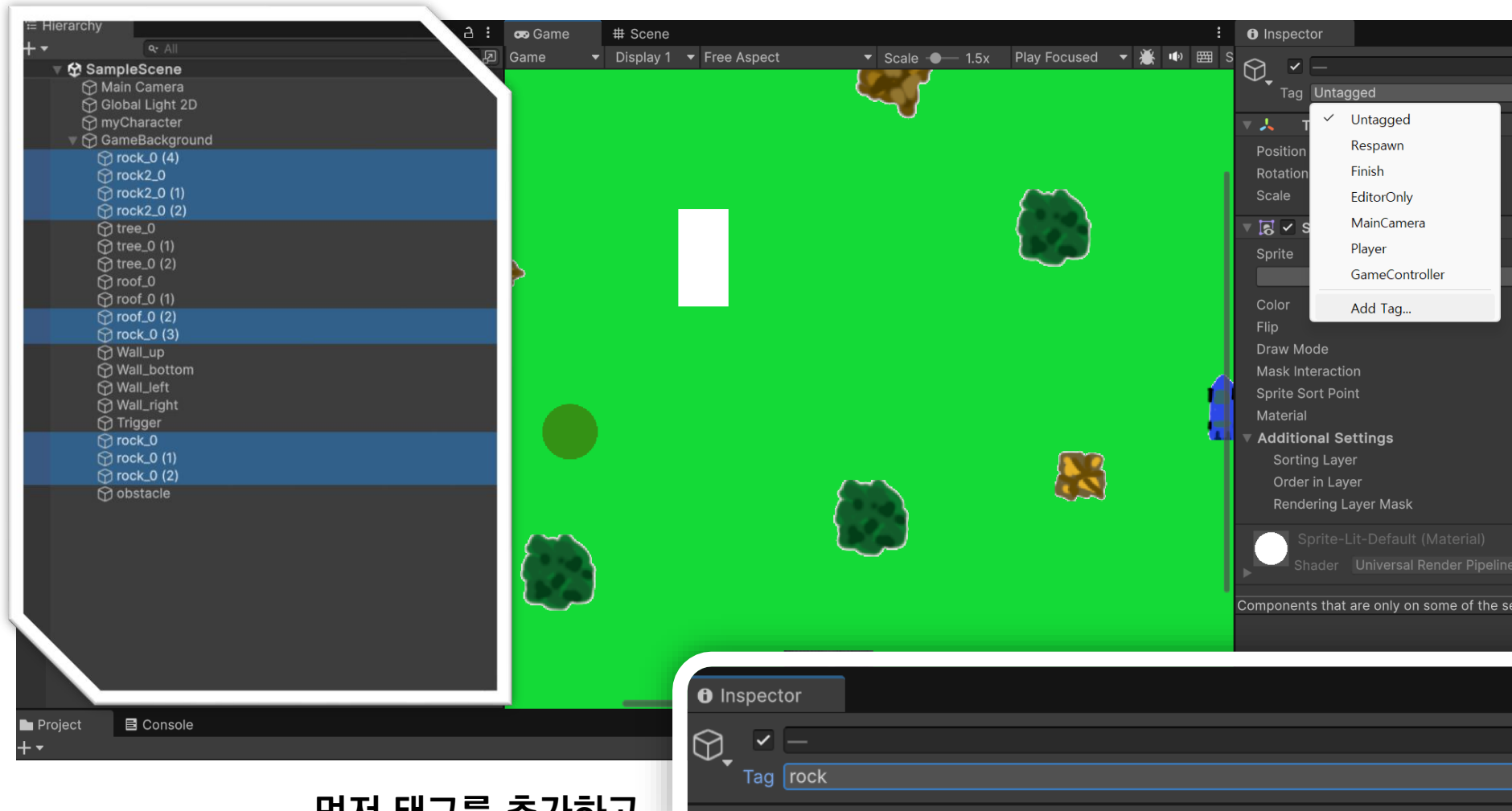
```
if ( something is true ) {  
    do this things  
}
```



```
if ( car hits a rock ) {  
    destroy the rock  
}
```

바위에 부딪힌 것을 알려면,
바위 객체들에게 같은 종류의
꼬리표(tag)를 달아 두자

Rock 객체들을 다중 선택하여 Add Tag...



먼저 태그를 추가하고,
객체들에 대해 태그를 지정한다

자동차와 바위가 충돌할 때 동작 구현

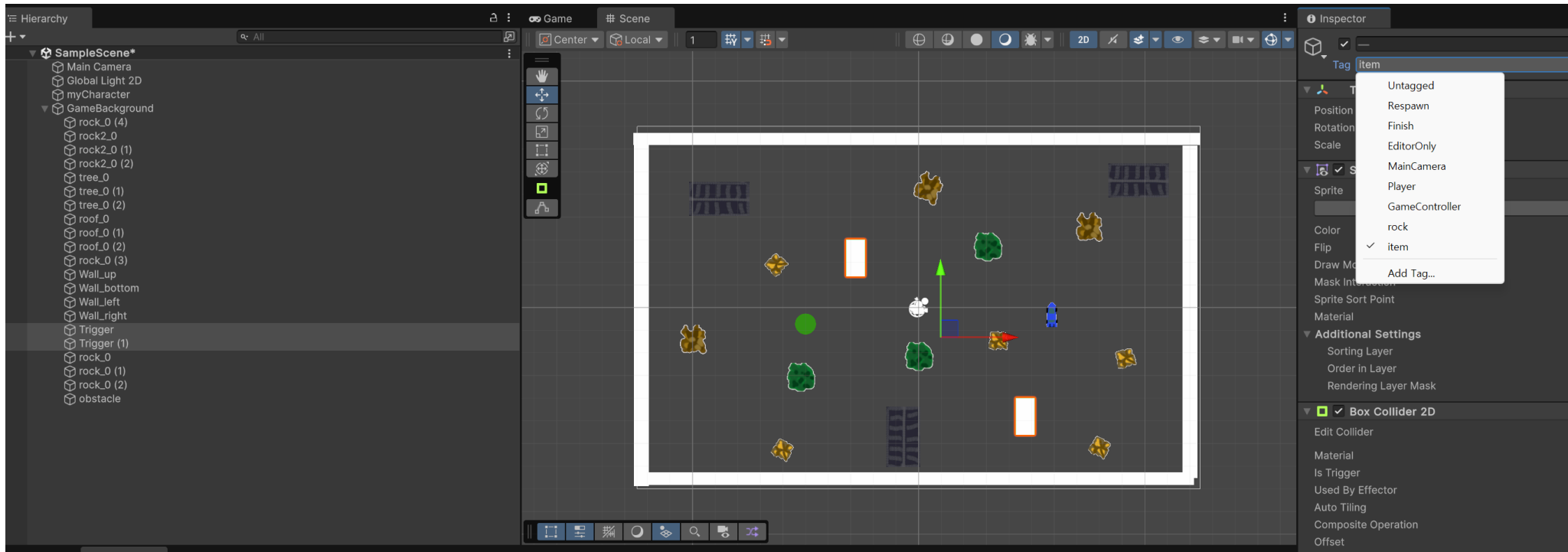
```
0 references
void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.gameObject.tag == "rock")
    {
        Debug.Log("Rock Collision!");
    }
}
```

Project Console

Clear Collapse Error Pause Editor

- [16:16:22] Rock Collision!
UnityEngine.Debug:Log (object)
- [16:16:23] Rock Collision!
UnityEngine.Debug:Log (object)
- [16:16:25] Rock Collision!
UnityEngine.Debug:Log (object)
- [16:16:26] Rock Collision!
UnityEngine.Debug:Log (object)
- [16:16:29] Rock Collision!
UnityEngine.Debug:Log (object)
- [16:16:31] Rock Collision!
UnityEngine.Debug:Log (object)
- [16:16:34] Rock Collision!
UnityEngine.Debug:Log (object)
- [16:16:35] Something has passed!
UnityEngine.Debug:Log (object)
- [16:16:39] Something has passed!
UnityEngine.Debug:Log (object)
- [16:16:40] Something has passed!
UnityEngine.Debug:Log (object)

Trigger 객체를 item으로 태그를 달아보자

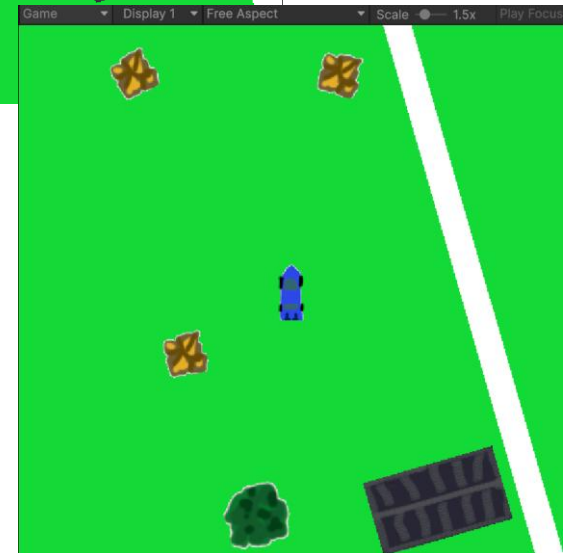
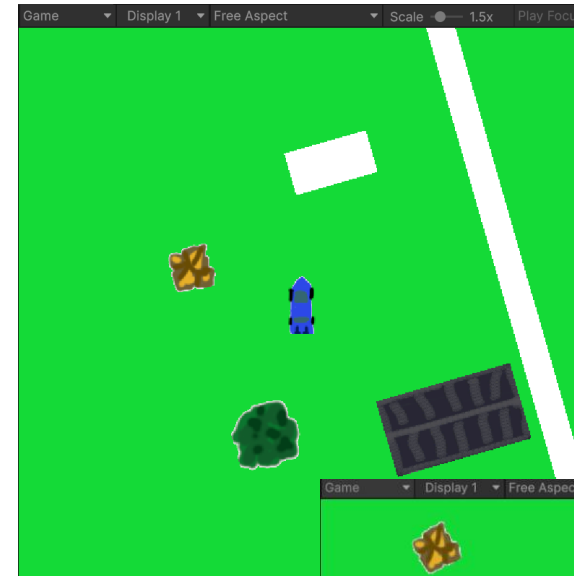


아이템에 도달하면 아이템이 사라지게 하자

```
public class myCharacter_move : MonoBehaviour
{
    1 reference
    [SerializeField] float steerSpeed = 100f;
    1 reference
    [SerializeField] float moveSpeed = 10f;
    // Start is called once before the first execution of Update after the MonoBehaviour is created
    0 references
    void Start()
    {
    }

    // Update is called once per frame
    0 references
    void Update()
    {
        float steerAmount = -Input.GetAxis("Horizontal") * Time.deltaTime;
        float moveAmount = Input.GetAxis("Vertical") * moveSpeed * Time.deltaTime;
        transform.Rotate(0, 0, steerSpeed * steerAmount);
        transform.Translate(0, moveAmount, 0);
    }

    0 references
    void OnCollisionEnter2D(Collision2D collision)
    {
        if (collision.gameObject.tag == "rock")
        {
            //Destroy(gameObject);
            Debug.Log("Rock Collision!");
        }
    }
}
```





이제 뭘 좀 만들어 볼까요?